

```

void TopologicalSort( struct Graph *G ) {
    struct Queue *Q;
    int counter;
    int v, w;
    Q = CreateQueue();
    counter = 0;
    for (v = 0; v < G→V; v++)
        if( indegree[v] == 0 )
            EnQueue( Q, v );
    while( !IsEmptyQueue( Q ) ) {
        v = DeQueue( Q );
        topologicalOrder[v] = ++counter;
        for each w adjacent to v
            if( --indegree[w] == 0 )
                EnQueue ( Q, w );
    }
    if( counter != G→V )
        printf("Graph has cycle");
    DeleteQueue( Q );
}

```

Total running time of topological sort is $O(V + E)$.

Note: The Topological sorting problem can be solved with DFS. Refer to the *Problems Section* for the algorithm.

Applications of Topological Sorting

- Representing course prerequisites
- Detecting deadlocks
- Pipeline of computing jobs
- Checking for symbolic link loop
- Evaluating formulae in spreadsheet

9.7 Shortest Path Algorithms

Let us consider the other important problem of a graph. Given a graph $G = (V, E)$ and a

distinguished vertex s , we need to find the shortest path from s to every other vertex in G . There are variations in the shortest path algorithms which depend on the type of the input graph and are given below.

Variations of Shortest Path Algorithms

Shortest path in unweighted graph
Shortest path in weighted graph
Shortest path in weighted graph with negative edges

Applications of Shortest Path Algorithms

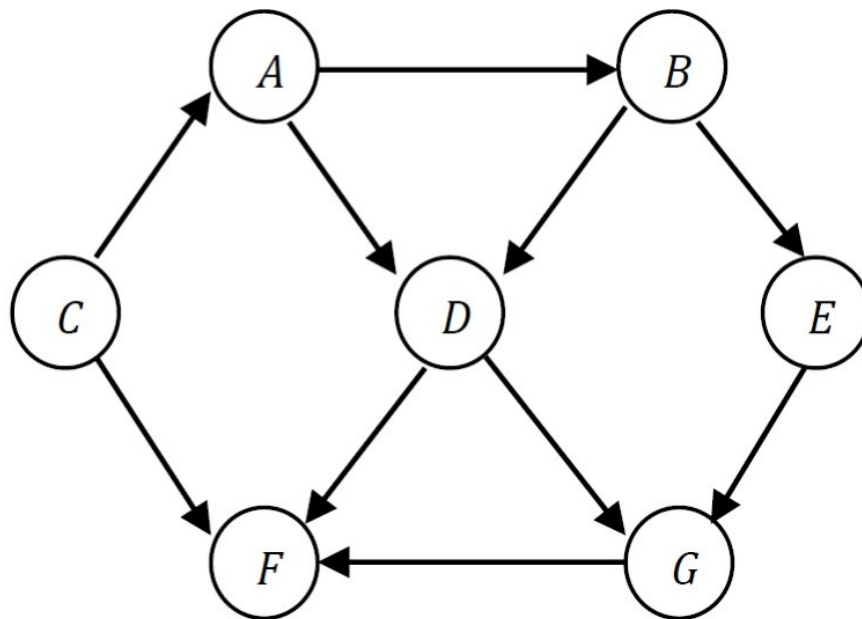
- Finding fastest way to go from one place to another
- Finding cheapest way to fly/send data from one city to another

Shortest Path in Unweighted Graph

Let s be the input vertex from which we want to find the shortest path to all other vertices. Unweighted graph is a special case of the weighted shortest-path problem, with all edges a weight of 1. The algorithm is similar to BFS and we need to use the following data structures:

- A distance table with three columns (each row corresponds to a vertex):
 - Distance from source vertex.
 - Path – contains the name of the vertex through which we get the shortest distance.
- A queue is used to implement breadth-first search. It contains vertices whose distance from the source node has been computed and their adjacent vertices are to be examined.

As an example, consider the following graph and its adjacency list representation.



The adjacency list for this graph is:

A: B → D

B: D → E

C: A → F

D: F → G

E: G

F: –

G: F

Let $s = C$. The distance from C to C is 0. Initially, distances to all other nodes are not computed, and we initialize the second column in the distance table for all vertices (except C) with -1 as below.

Vertex	Distance[v]	Previous vertex which gave Distance[v]
<i>A</i>	-1	-
<i>B</i>	-1	-
<i>C</i>	0	-
<i>D</i>	-1	-
<i>E</i>	-1	-
<i>F</i>	-1	-
<i>G</i>	-1	-

Algorithm

```

void UnweightedShortestPath(struct Graph *G, int s) {
    struct Queue *Q = CreateQueue();
    int v, w;
    EnQueue(Q, s);
    for (int i = 0; i < G→V; i++)
        Distance[i] = -1;
    Distance[s] = 0;
    while (!IsEmptyQueue(Q)) {
        v = DeQueue(Q);
        for each w adjacent to v
            if (Distance[w] == -1) {
                Distance[w] = Distance[v] + 1;
                Path[w] = v;
                EnQueue(Q, w);
            }
    }
    DeleteQueue(Q);
}

```

Each vertex examined at most once

Each vertex EnQueue'd at most once

Running time: $O(|E| + |V|)$, if adjacency lists are used. In for loop, we are checking the outgoing edges for a given vertex and the sum of all examined edges in the while loop is equal to the number of edges which gives $O(|E|)$.

If we use matrix representation the complexity is $O(|V|^2)$, because we need to read an entire row in the matrix of length $|V|$ in order to find the adjacent vertices for a given vertex.

Shortest path in Weighted Graph [Dijkstra's]

A famous solution for the shortest path problem was developed by *Dijkstra*. *Dijkstra's* algorithm is a generalization of the BFS algorithm. The regular BFS algorithm cannot solve the shortest path problem as it cannot guarantee that the vertex at the front of the queue is the vertex closest to source s .

Before going to code let us understand how the algorithm works. As in unweighted shortest path algorithm, here too we use the distance table. The algorithm works by keeping the shortest distance of vertex v from the source in the *Distance* table. The value *Distance*[v] holds the distance from s to v . The shortest distance of the source to itself is zero. The *Distance* table for all other vertices is set to -1 to indicate that those vertices are not already processed.

Vertex	Distance[v]	Previous vertex which gave Distance[v]
A	-1	-
B	-1	-
C	0	-
D	-1	-
E	-1	-
F	-1	-
G	-1	-

After the algorithm finishes, the *Distance* table will have the shortest distance from source s to each other vertex v . To simplify the understanding of *Dijkstra's* algorithm, let us assume that the given vertices are maintained in two sets. Initially the first set contains only the source element and the second set contains all the remaining elements. After the k^{th} iteration, the first set contains k vertices which are closest to the source. These k vertices are the ones for which we have already computed the shortest distances from source.

Notes on Dijkstra's Algorithm

- It uses greedy method: Always pick the next closest vertex to the source.
- It uses priority queue to store unvisited vertices by distance from s .
- It does not work with negative weights.

Difference between Unweighted Shortest Path and Dijkstra's Algorithm

- 1) To represent weights in the adjacency list, each vertex contains the weights of the edges (in addition to their identifier).
- 2) Instead of ordinary queue we use priority queue [distances are the priorities] and the vertex with the smallest distance is selected for processing.
- 3) The distance to a vertex is calculated by the sum of the weights of the edges on the path from the source to that vertex.
- 4) We update the distances in case the newly computed distance is smaller than the old distance which we have already computed.

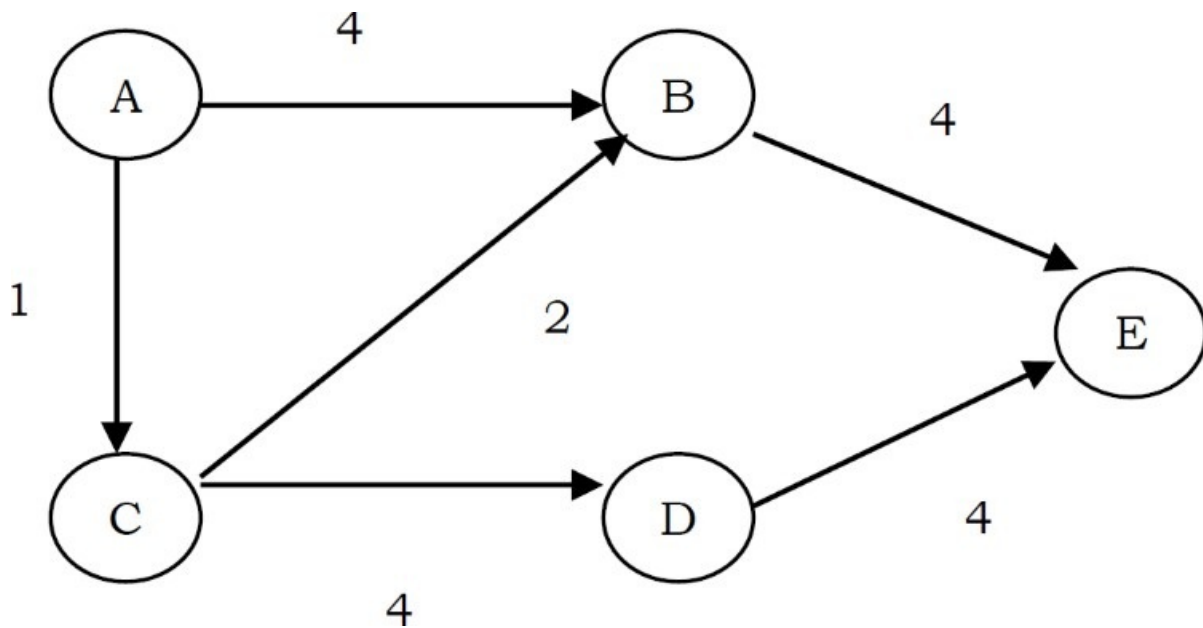
```

void Dijkstra(struct Graph *G, int s) {
    struct PriorityQueue *PQ = CreatePriorityQueue();
    int v, w;
    EnQueue(PQ, s);
    for (int i = 0; i < G→V; i++)
        Distance[i] = -1;
    Distance[s] = 0;
    while (!IsEmptyQueue(PQ)) {
        v = DeleteMin(PQ);
        for all adjacent vertices w of v {
            Compute new distance d = Distance[v] + weight[v][w];
            if (Distance[w] == -1) {
                Distance[w] = new distance d;
                Insert w in the priority queue with priority d
                Path[w] = v;
            }
            if (Distance[w] > new distance d) {
                Distance[w] = new distance d;
                Update priority of vertex w to be d;
                Path[w] = v;
            }
        }
    }
}

```

The above algorithm can be better understood through an example, which will explain each step that is taken and how *Distance* is calculated. The weighted graph below has 5 vertices from A – E.

The value between the two vertices is known as the edge cost between two vertices. For example, the edge cost between A and C is 1. Dijkstra's algorithm can be used to find the shortest path from source A to the remaining vertices in the graph.

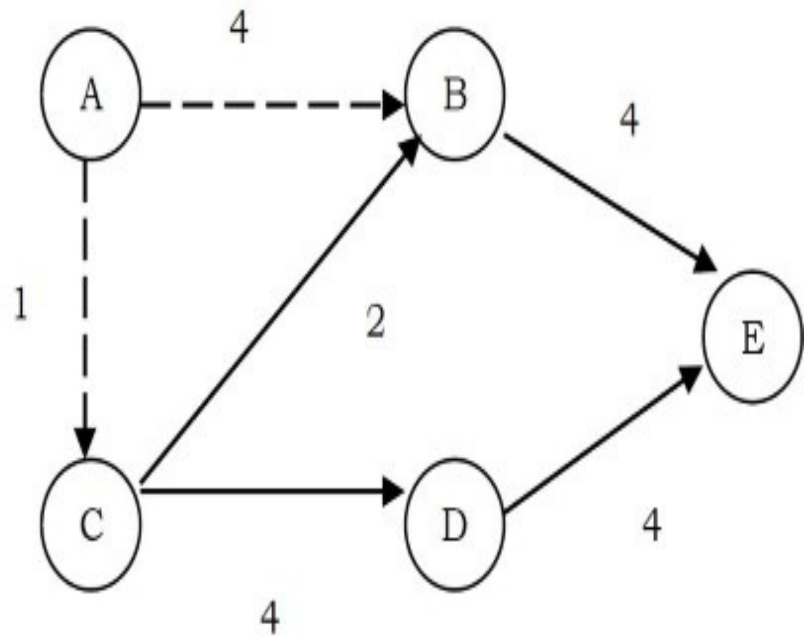


Initially the *Distance* table is:

Vertex	Distance[v]	Previous vertex which gave Distance[v]
<i>A</i>	0	-
<i>B</i>	-1	-
<i>C</i>	-1	-
<i>D</i>	-1	-
<i>E</i>	-1	-

After the first step, from vertex *A*, we can reach *B* and *C*. So, in the *Distance* table we update the reachability of *B* and *C* with their costs and the same is shown below.

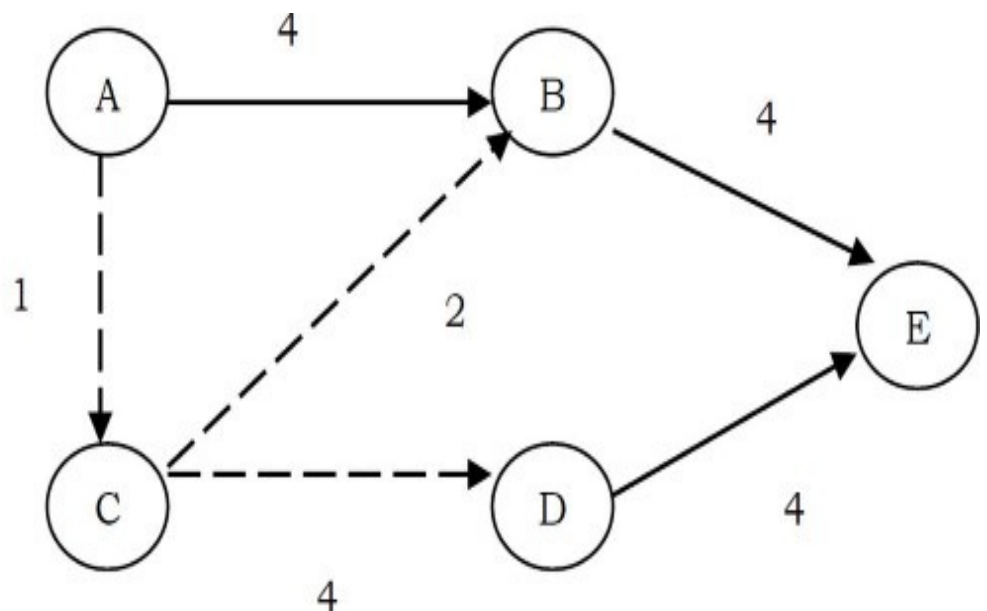
A	0	-
B	4	A
C	1	A
D	-1	-
E	-1	-



Shortest path from B, C from A

Now, let us select the minimum distance among all. The minimum distance vertex is C. That means, we have to reach other vertices from these two vertices (A and C). For example, B can be reached from A and also from C. In this case we have to select the one which gives the lowest cost. Since reaching B through C is giving the minimum cost ($1 + 2$), we update the *Distance* table for vertex B with cost 3 and the vertex from which we got this cost as C.

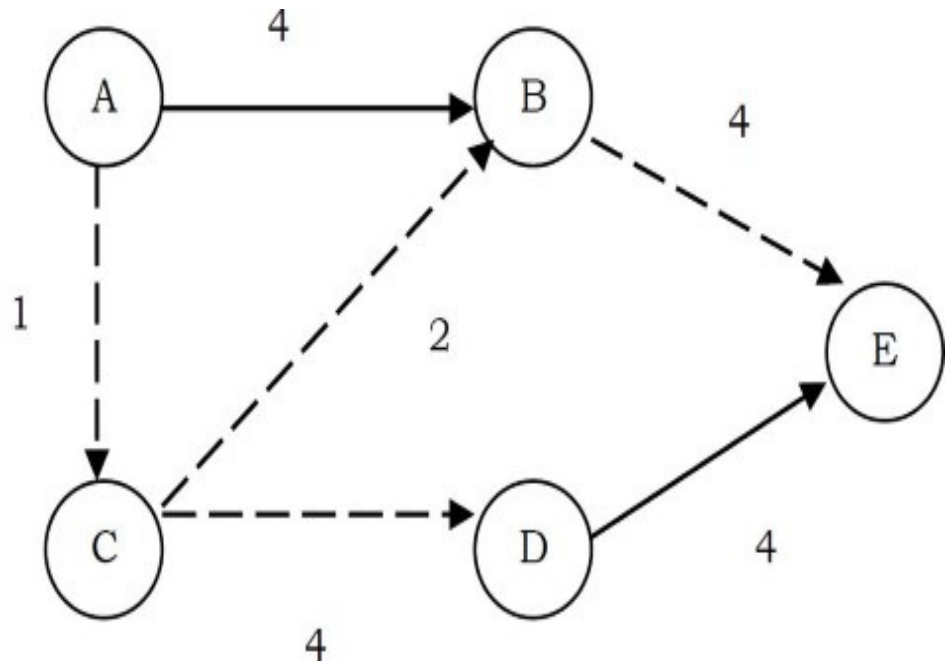
A	0	-
B	3	C
C	1	A
D	5	C
E	-1	-



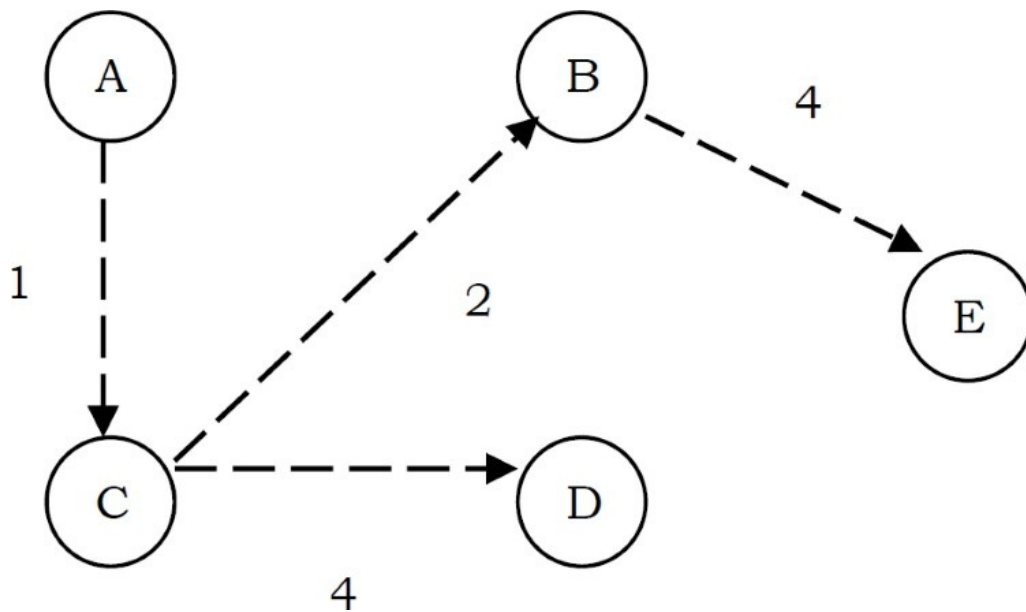
Shortest path to B, D using C as intermediate vertex

The only vertex remaining is E. To reach E, we have to see all the paths through which we can reach E and select the one which gives the minimum cost. We can see that if we use B as the intermediate vertex through C we get the minimum cost.

A	0	-
B	3	C
C	1	A
D	5	C
E	7	B



The final minimum cost tree which Dijkstra's algorithm generates is:



Performance

In Dijkstra's algorithm, the efficiency depends on the number of DeleteMins (V DeleteMins) and updates for priority queues (E updates) that are used. If a *standard binary heap* is used then the complexity is $O(E \log V)$.

The term $E \log V$ comes from E updates (each update takes $\log V$) for the standard heap. If the set used is an array then the complexity is $O(E + V^2)$.

Disadvantages of Dijkstra's Algorithm

- As discussed above, the major disadvantage of the algorithm is that it does a blind search, thereby wasting time and necessary resources.
- Another disadvantage is that it cannot handle negative edges. This leads to acyclic graphs and most often cannot obtain the right shortest path.

Relatives of Dijkstra's Algorithm

- The *Bellman-Ford* algorithm computes single-source shortest paths in a weighted digraph. It uses the same concept as that of *Dijkstra's* algorithm but can handle negative edges as well. It has more running time than *Dijkstra's* algorithm.
- Prim's algorithm finds a minimum spanning tree for a connected weighted graph. It implies that a subset of edges that form a tree where the total weight of all the edges in the tree is minimized.

Bellman-Ford Algorithm

If the graph has negative edge costs, then *Dijkstra's* algorithm does not work. The problem is that once a vertex u is declared known, it is possible that from some other, unknown vertex v there is a path back to u that is very negative. In such a case, taking a path from s to v back to u is better than going from s to u without using v . A combination of Dijkstra's algorithm and unweighted algorithms will solve the problem. Initialize the queue with s . Then, at each stage, we *DeQueue* a vertex v . We find all vertices W adjacent to v such that,

$$\text{distance to } v + \text{weight}(v,w) < \text{old distance to } w$$

We update w old distance and path, and place w on a queue if it is not already there. A bit can be set for each vertex to indicate presence in the queue. We repeat the process until the queue is empty.

```

void BellmanFordAlgorithm(struct Graph *G, int s) {
    struct Queue *Q = CreateQueue();
    int v, w;
    EnQueue(Q, s);
    Distance[s] = 0;           // assume the Distance table is filled with INT_MAX
    while (!IsEmptyQueue(Q)) {
        v = DeQueue(Q);
        for all adjacent vertices w of v {
            Compute new distance d = Distance[v] + weight[v][w];
            if (old distance to w > new distance d) {
                Distance[w] = (distance to v) + weight[v][w];
                Path[w] = v;
                if (w is there in queue)
                    EnQueue(Q, w)
            }
        }
    }
}

```

This algorithm works if there are no negative-cost cycles. Each vertex can DeQueue at most $|V|$ times, so the running time is $O(|E| \cdot |V|)$ if adjacency lists are used.

Overview of Shortest Path Algorithms

Shortest path in unweighted graph [<i>Modified BFS</i>]	$O(E + V)$
Shortest path in weighted graph [<i>Dijkstra's</i>]	$O(E \log V)$
Shortest path in weighted graph with negative edges [<i>Bellman – Ford</i>]	$O(E \cdot V)$
Shortest path in weighted acyclic graph	$O(E + V)$

9.8 Minimal Spanning Tree

The *Spanning tree* of a graph is a subgraph that contains all the vertices and is also a tree. A graph may have many spanning trees. As an example, consider a graph with 4 vertices as shown below. Let us assume that the corners of the graph are vertices.



For this simple graph, we can have multiple spanning trees as shown below.



The algorithm we will discuss now is *minimum spanning tree* in an undirected graph. We assume that the given graphs are weighted graphs. If the graphs are unweighted graphs then we can still use the weighted graph algorithms by treating all weights as equal. A *minimum spanning tree* of an undirected graph G is a tree formed from graph edges that connect all the vertices of G with minimum total cost (weights). A minimum spanning tree exists only if the graph is connected. There are two famous algorithms for this problem:

- *Prim's Algorithm*
- *Kruskal's Algorithm*

Prim's Algorithm

Prim's algorithm is almost the same as Dijkstra's algorithm. As in Dijkstra's algorithm, in Prim's algorithm we keep the values *distance* and *paths* in the distance table. The only exception is that since the definition of *distance* is different, the updating statement also changes a little. The update statement is simpler than before.

```

void Prims(struct Graph *G, int s) {
    struct PriorityQueue *PQ = CreatePriorityQueue();
    int v, w;
    EnQueue(PQ, s);
    Distance[s] = 0;           // assume the Distance table is filled with -1
    while (!IsEmptyQueue(PQ)) {
        v = DeleteMin(PQ);
        for all adjacent vertices w of v {
            Compute new distance d= Distance[v] + weight[v][w];
            if(Distance[w] == -1) {
                Distance[w] = weight[v][w];
                Insert w in the priority queue with priority d
                Path[w] = v;
            }
            if(Distance[w] > new distance d) {
                Distance[w] = weight[v][w];
                Update priority of vertex w to be d;
                Path[w] = v;
            }
        }
    }
}

```

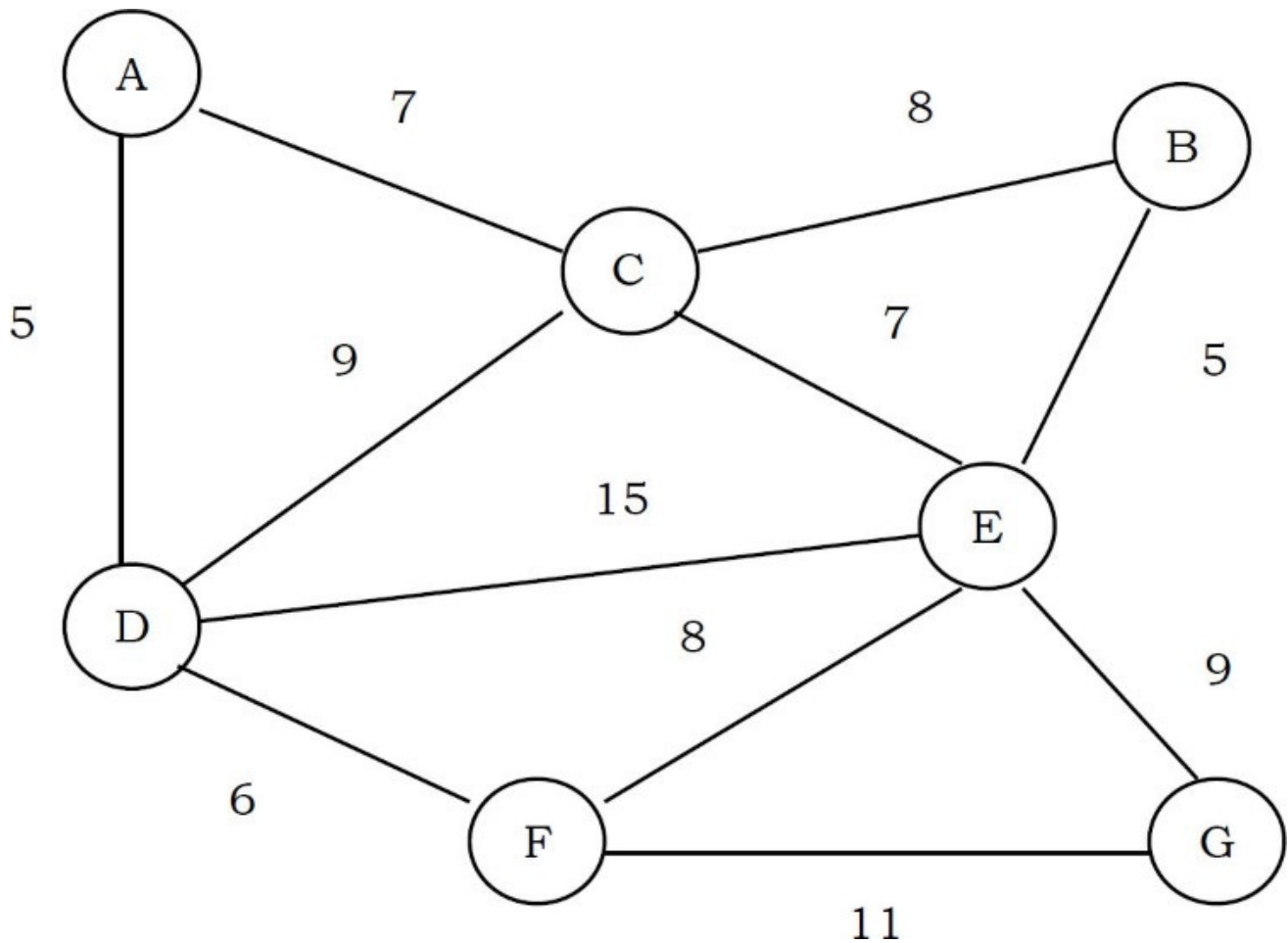
The entire implementation of this algorithm is identical to that of Dijkstra's algorithm. The running time is $O(|V|^2)$ without heaps [good for dense graphs], and $O(E \log V)$ using binary heaps [good for sparse graphs].

Kruskal's Algorithm

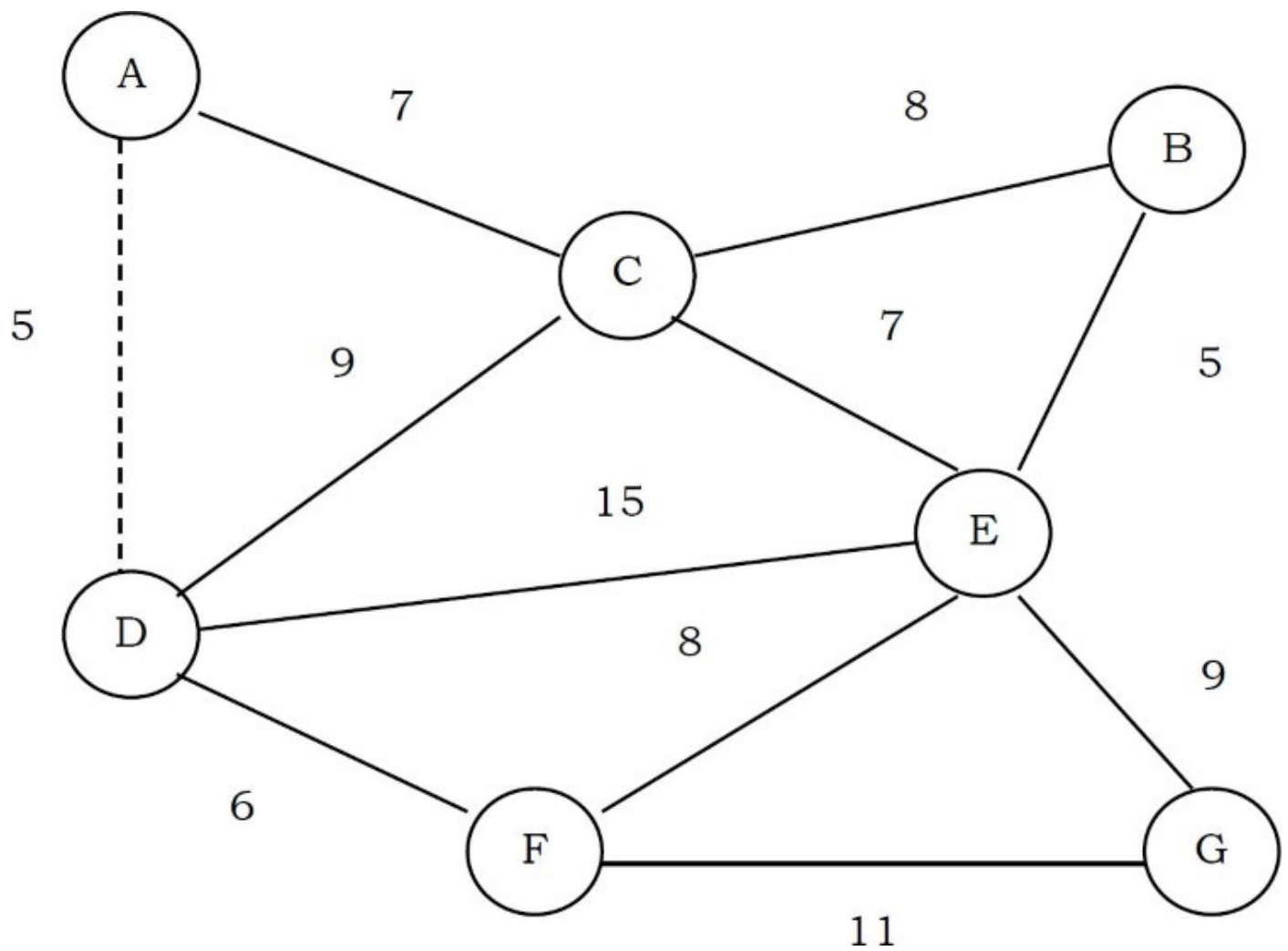
The algorithm starts with V different trees (V is the vertices in the graph). While constructing the minimum spanning tree, every time Kruskal's algorithm selects an edge that has minimum weight and then adds that edge if it doesn't create a cycle. So, initially, there are $|V|$ single-node trees in the forest. Adding an edge merges two trees into one. When the algorithm is completed, there will be only one tree, and that is the minimum spanning tree. There are two ways of implementing Kruskal's algorithm:

- By using Disjoint Sets: Using UNION and FIND operations
- By using Priority Queues: Maintains weights in priority queue

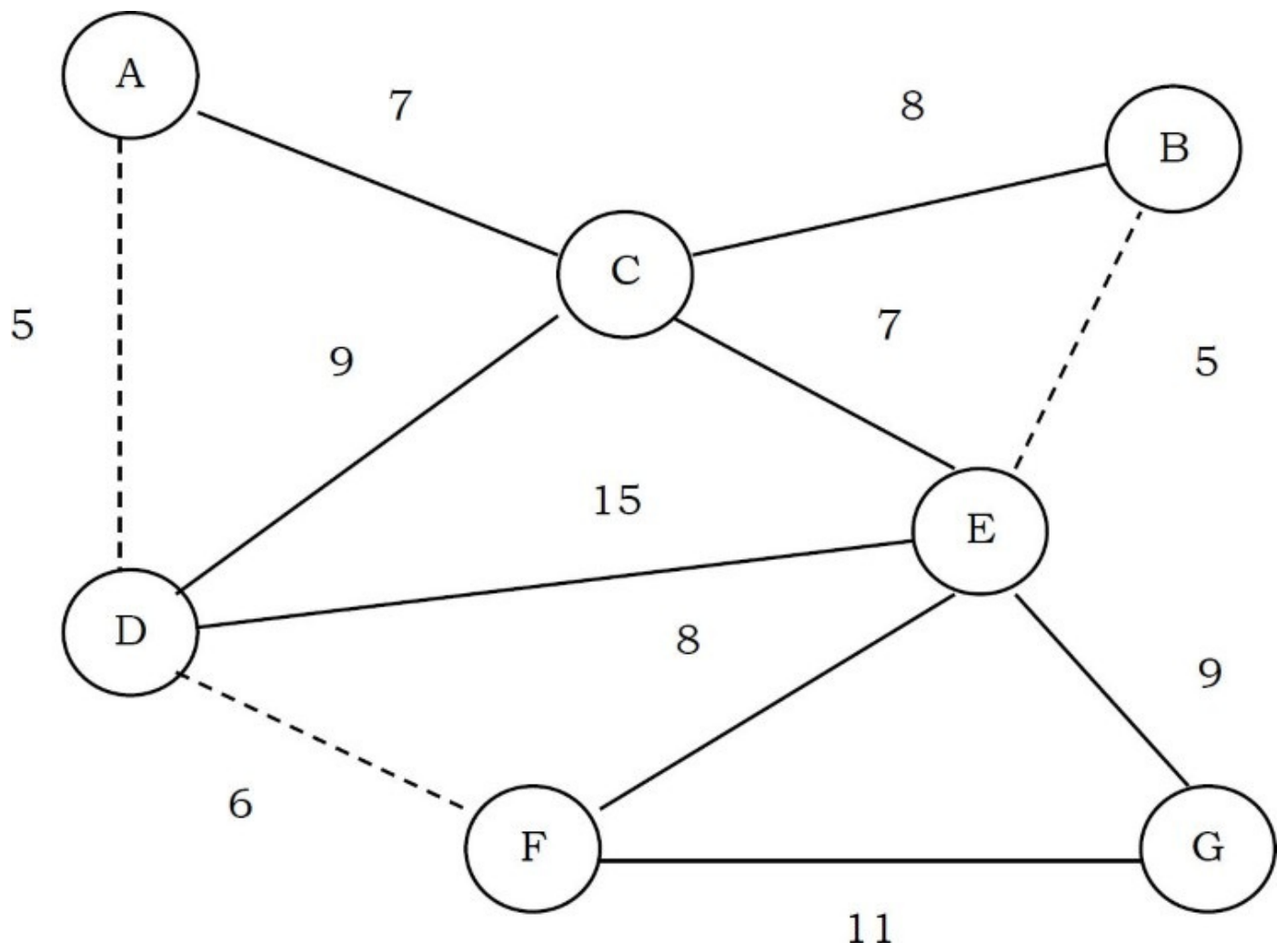
The appropriate data structure is the UNION/FIND algorithm [for implementing forests]. Two vertices belong to the same set if and only if they are connected in the current spanning forest. Each vertex is initially in its own set. If u and v are in the same set, the edge is rejected because it forms a cycle. Otherwise, the edge is accepted, and a UNION is performed on the two sets containing u and v . As an example, consider the following graph (the edges show the weights).



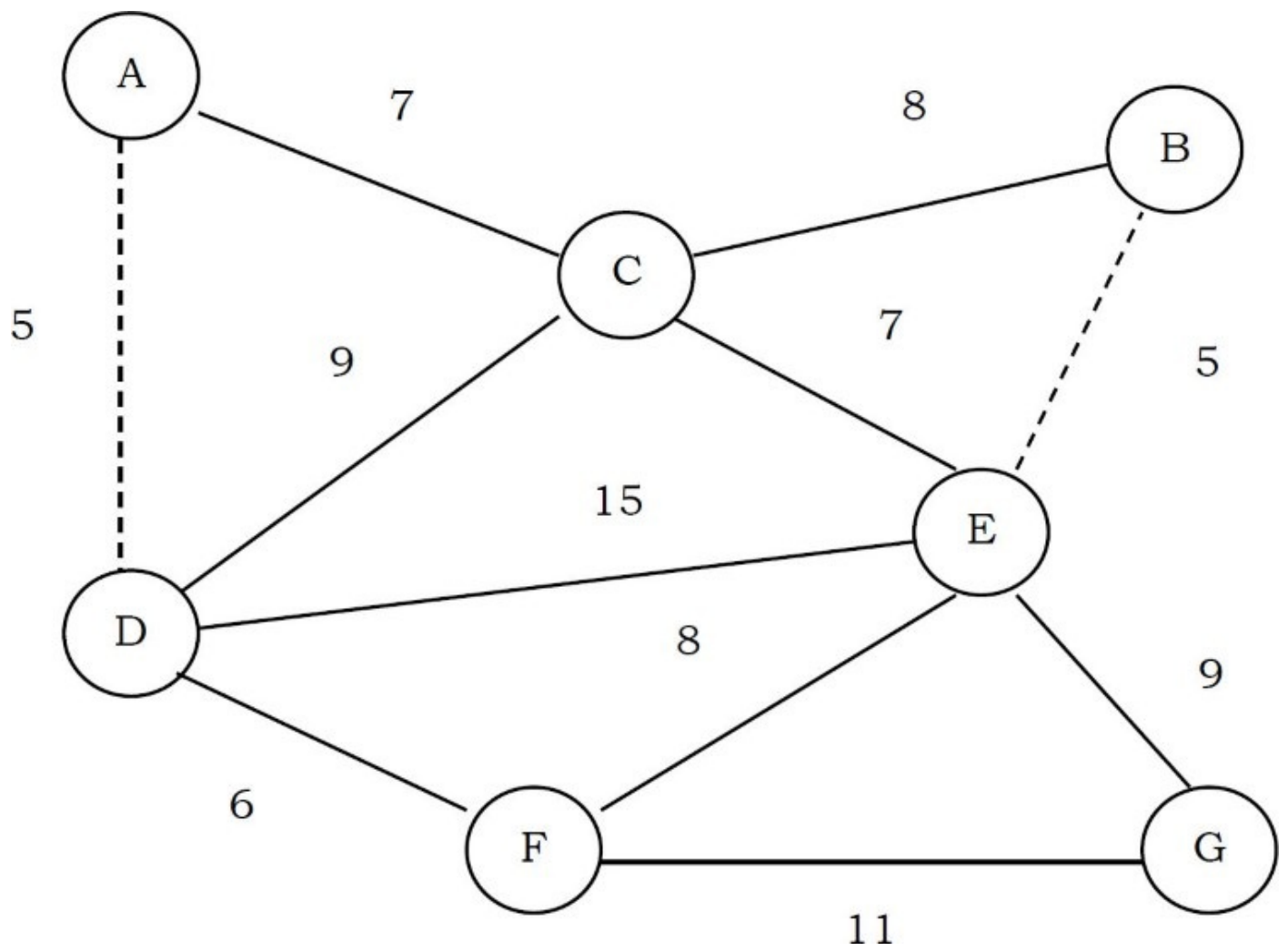
Now let us perform Kruskal's algorithm on this graph. We always select the edge which has minimum weight.



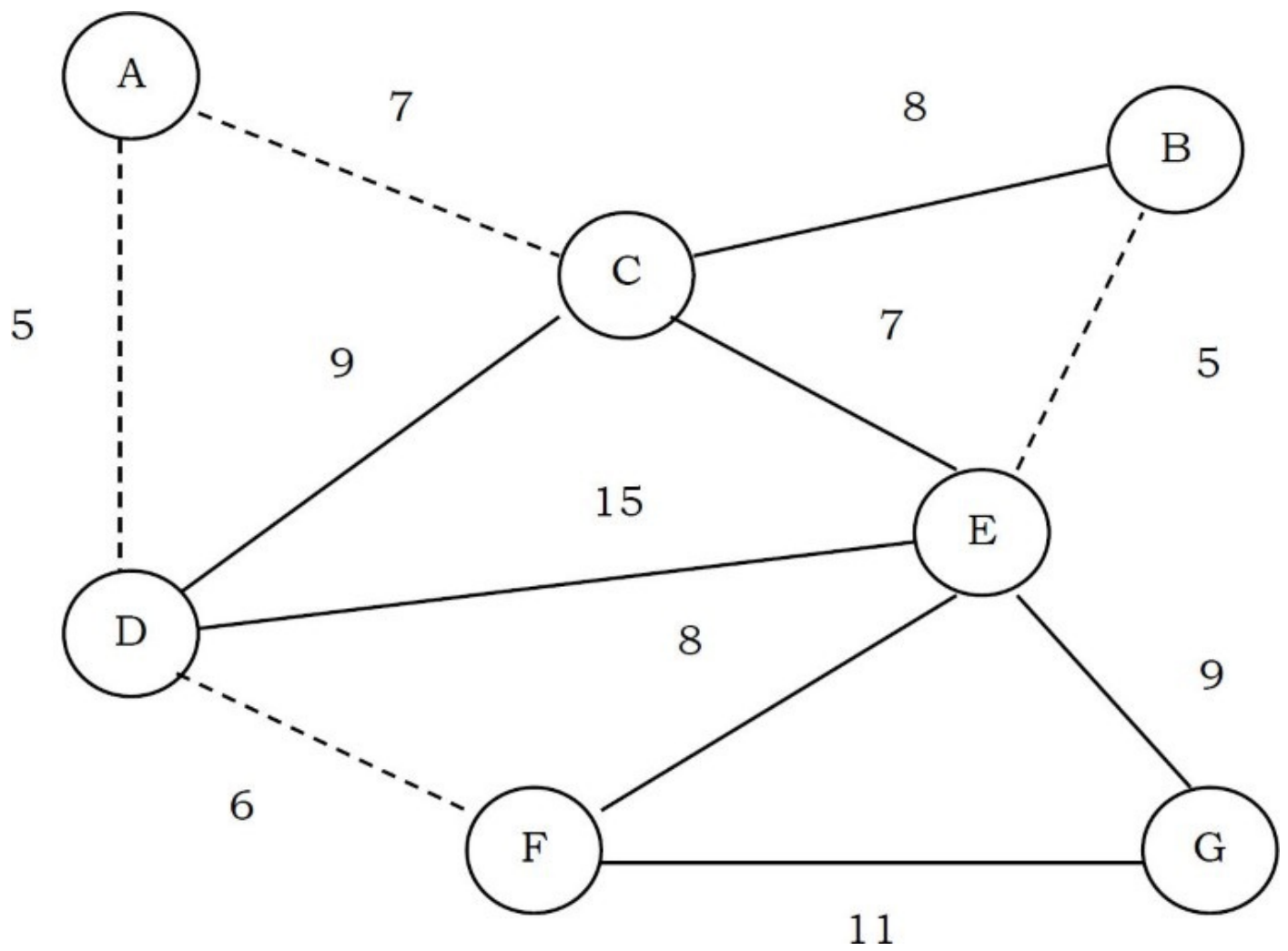
From the above graph, the edges which have minimum weight (cost) are: AD and BE. From these two we can select one of them and let us assume that we select AD (dotted line).



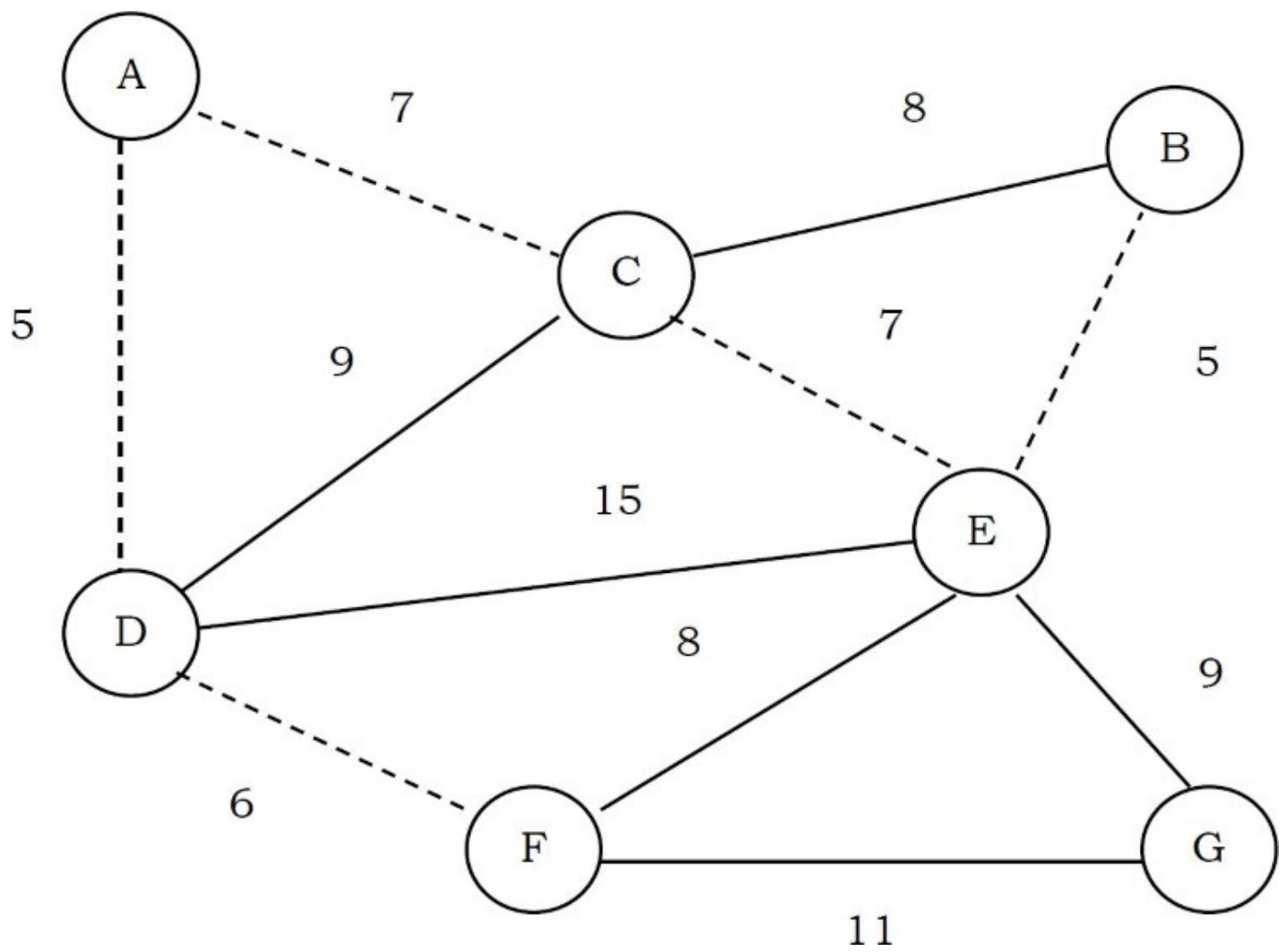
DF is the next edge that has the lowest cost (6).



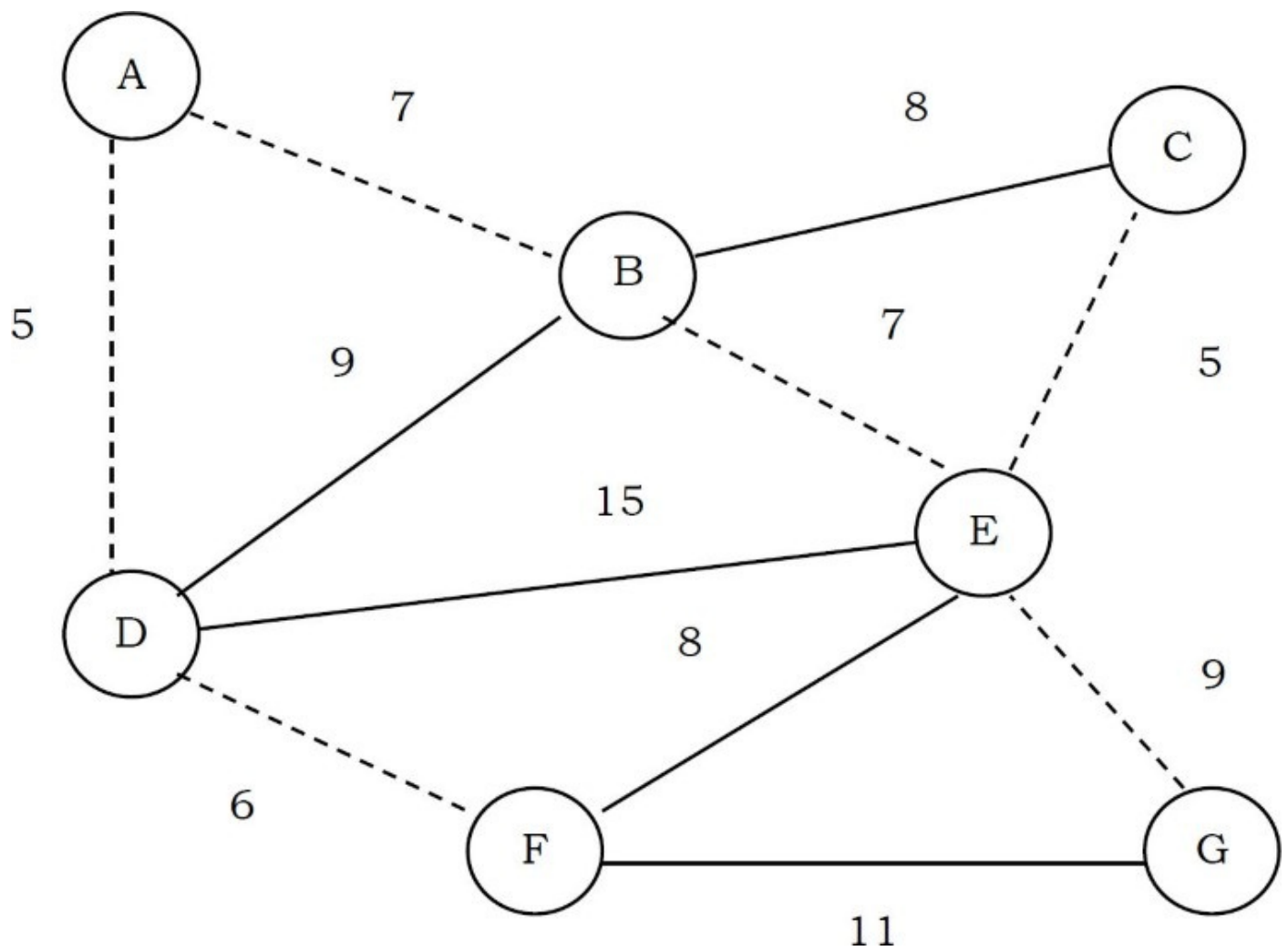
BE now has the lowest cost and we select it (dotted lines indicate selected edges).



Next, AC and CE have the low cost of 7 and we select AC.



Then we select CE as its cost is 7 and it does not form a cycle.



The next low cost edges are CB and EF. But if we select CB, then it forms a cycle. So we discard it. This is also the case with EF. So we should not select those two. And the next low cost is 9 (BD and EG). Selecting BD forms a cycle so we discard it. Adding EG will not form a cycle and therefore with this edge we complete all vertices of the graph.

```

void Kruskal(struct Graph *G) {
    S =  $\phi$ ; // At the end S will contains the edges of minimum spanning trees
    for (int v = 0; v < G→V; v++)
        MakeSet (v);
    Sort edges of E by increasing weights w;
    for each edge (u, v) in E { //from sorted list
        if(FIND (u)  $\neq$  FIND (v)) {
            S = S  $\cup$  {(u, v)};
            UNION (u, v);
        }
    }
    return S;
}

```

Note: For implementation of UNION and FIND operations, refer to the [Disjoint Sets ADT](#) chapter.

The worst-case running time of this algorithm is $O(E \log E)$, which is dominated by the heap operations. That means, since we are constructing the heap with E edges, we need $O(E \log E)$ time to do that.

9.9 Graph Algorithms: Problems & Solutions

Problem-1 In an undirected simple graph with n vertices, what is the maximum number of edges? Self-loops are not allowed.

Solution: Since every node can connect to all other nodes, the first node can connect to $n - 1$ nodes. The second node can connect to $n - 2$ nodes [since one edge is already there from the first node]. The total number of edges is: $1 + 2 + 3 + \dots + n - 1 = \frac{n(n-1)}{2}$ edges.

Problem-2 How many different adjacency matrices does a graph with n vertices and E edges have?

Solution: It's equal to the number of permutations of n elements, i.e., $n!$.

Problem-3 How many different adjacency lists does a graph with n vertices have?

Solution: It's equal to the number of permutations of edges, i.e., $E!$.

Problem-4 Which undirected graph representation is most appropriate for determining whether or not a vertex is isolated (is not connected to any other vertex)?

Solution: Adjacency List. If we use the adjacency matrix, then we need to check the complete row to determine whether that vertex has edges or not. By using the adjacency list, it is very easy to check, and it can be done just by checking whether that vertex has NULL for next pointer or not [NULL indicates that the vertex is not connected to any other vertex].

Problem-5 For checking whether there is a path from source s to target t , which one is best between disjoint sets and DFS?

Solution: The table below shows the comparison between disjoint sets and DFS. The entries in the table represent the case for any pair of nodes (for s and t).

Method	Processing Time	Query Time	Space
Union-Find	$V + E \log V$	$\log V$	V
DFS	$E + V$	1	$E + V$

Problem-6 What is the maximum number of edges a directed graph with n vertices can have and still not contain a directed cycle?

Solution: The number is $V(V-1)/2$. Any directed graph can have at most n^2 edges. However, since the graph has no cycles it cannot contain a self loop, and for any pair x,y of vertices, at most one edge from (x,y) and (y,x) can be included. Therefore the number of edges can be at most $(V^2 - V)/2$ as desired. It is possible to achieve $V(V-1)/2$ edges. Label n nodes $1,2,\dots,n$ and add an edge (x,y) if and only if $x < y$. This graph has the appropriate number of edges and cannot contain a cycle (any path visits an increasing sequence of nodes).

Problem-7 How many simple directed graphs with no parallel edges and self-loops are possible in terms of V ?

Solution: $(V) \times (V-1)$. Since, each vertex can connect to $V-1$ vertices without self-loops.

Problem-8 What are the differences between DFS and BFS?

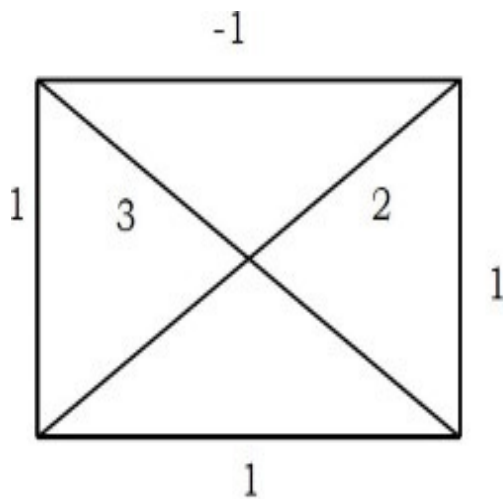
Solution:

DFS	BFS
Backtracking is possible from a dead end.	Backtracking is not possible.
Vertices from which exploration is incomplete are processed in a LIFO order	The vertices to be explored are organized as a FIFO queue.
The search is done in one particular direction	The vertices at the same level are maintained in parallel.

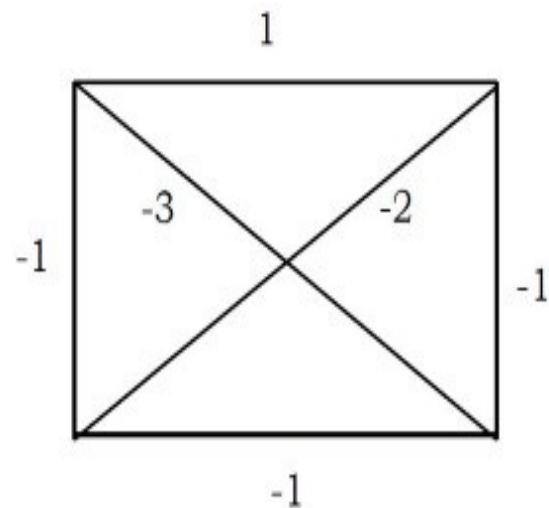
Problem-9 Earlier in this chapter, we discussed minimum spanning tree algorithms. Now,

give an algorithm for finding the maximum-weight spanning tree in a graph.

Solution:



Given graph



Transformed graph with negative edge weights

Using the given graph, construct a new graph with the same nodes and edges. But instead of using the same weights, take the negative of their weights. That means, weight of an edge = negative of weight of the corresponding edge in the given graph. Now, we can use existing *minimum spanning tree* algorithms on this new graph. As a result, we will get the maximum-weight spanning tree in the original one.

Problem-10 Give an algorithm for checking whether a given graph G has simple path from source s to destination d . Assume the graph G is represented using the adjacent matrix.

Solution: Let us assume that the structure for the graph is:

```
struct Graph {  
    int V;           //Number of vertices  
    int E;           //Number of edges  
    int ** adjMatrix; //Two dimensional array for storing the connections  
};
```

For each vertex call *DFS* and check whether the current vertex is the same as the destination vertex or not. If they are the same, then return 1. Otherwise, call the *DFS* on its unvisited neighbors. One important thing to note here is that, we are calling the *DFS* algorithm on vertices which are not yet visited.

```

void HasSimplePath(struct Graph *G, int s, int d) {
    int t;
    Viisited[s] = 1;
    if(s == d)
        return 1;
    for(t = 0; t < G→V; t++) {
        if(G→adjMatrix[s][t] && !Viisited[t])
            if(DFS(G, t, d))
                return 1;
    }
    return 0;
}

```

Time Complexity: $O(E)$. In the above algorithm, for each node, since we are not calling *DFS* on all of its neighbors (discarding through *if* condition), Space Complexity: $O(V)$.

Problem-11 Count simple paths for a given graph G has simple path from source s to destination d ? Assume the graph is represented using the adjacent matrix.

Solution: Similar to the discussion in [Problem-10](#), start at one node and call *DFS* on that node. As a result of this call, it visits all the nodes that it can reach in the given graph. That means it visits all the nodes of the connected component of that node. If there are any nodes that have not been visited, then again start at one of those nodes and call *DFS*.

Before the first *DFS* in each connected component, increment the connected components *count*. Continue this process until all of the graph nodes are visited. As a result, at the end we will get the total number of connected components. The implementation based on this logic is given below:

```

void CountSimplePaths(struct Graph * G, int s, int d) {
    int t;
    Visited[s] = 1;
    if(s == d) {
        count++;
        Visited[s] = 0;
        return;
    }
    for(t = 0; t < G->V; t++) {
        if(G->adjMatrix[s][t] && !Visited[t]) {
            DFS(G, t, d);
            Visited[t] = 0;
        }
    }
}

```

Problem-12 All pairs shortest path problem: Find the shortest graph distances between every pair of vertices in a given graph. Let us assume that the given graph does not have negative edges.

Solution: The problem can be solved using n applications of *Dijkstra's* algorithm. That means we apply *Dijkstra's* algorithm on each vertex of the given graph. This algorithm does not work if the graph has edges with negative weights.

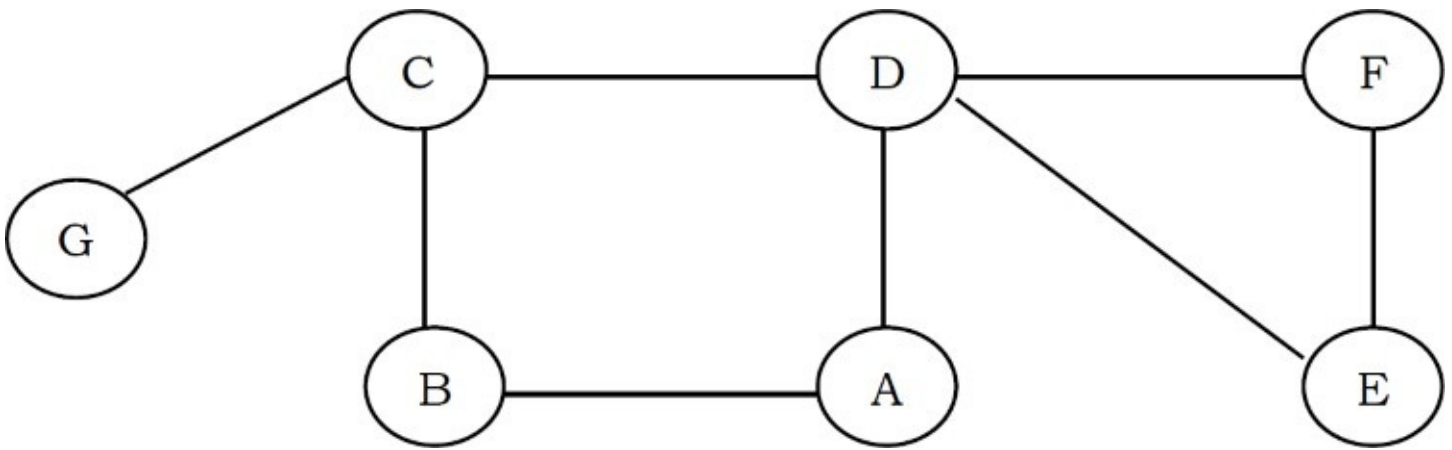
Problem-13 In [Problem-12](#), how do we solve the all pairs shortest path problem if the graph has edges with negative weights?

Solution: This can be solved by using the *Floyd – Warshall algorithm*. This algorithm also works in the case of a weighted graph where the edges have negative weights. This algorithm is an example of Dynamic Programming -refer to the *Dynamic Programming* chapter.

Problem-14 DFS Application: Cut Vertex or Articulation Points

Solution: In an undirected graph, a *cut vertex* (or articulation point) is a vertex, and if we remove it, then the graph splits into two disconnected components. As an example, consider the following figure. Removal of the “D” vertex divides the graph into two connected components ($\{E, F\}$ and $\{A, B, C, G\}$).

Similarly, removal of the “C” vertex divides the graph into ($\{G\}$ and $\{A, B, D, E, F\}$). For this graph, A and C are the cut vertices.

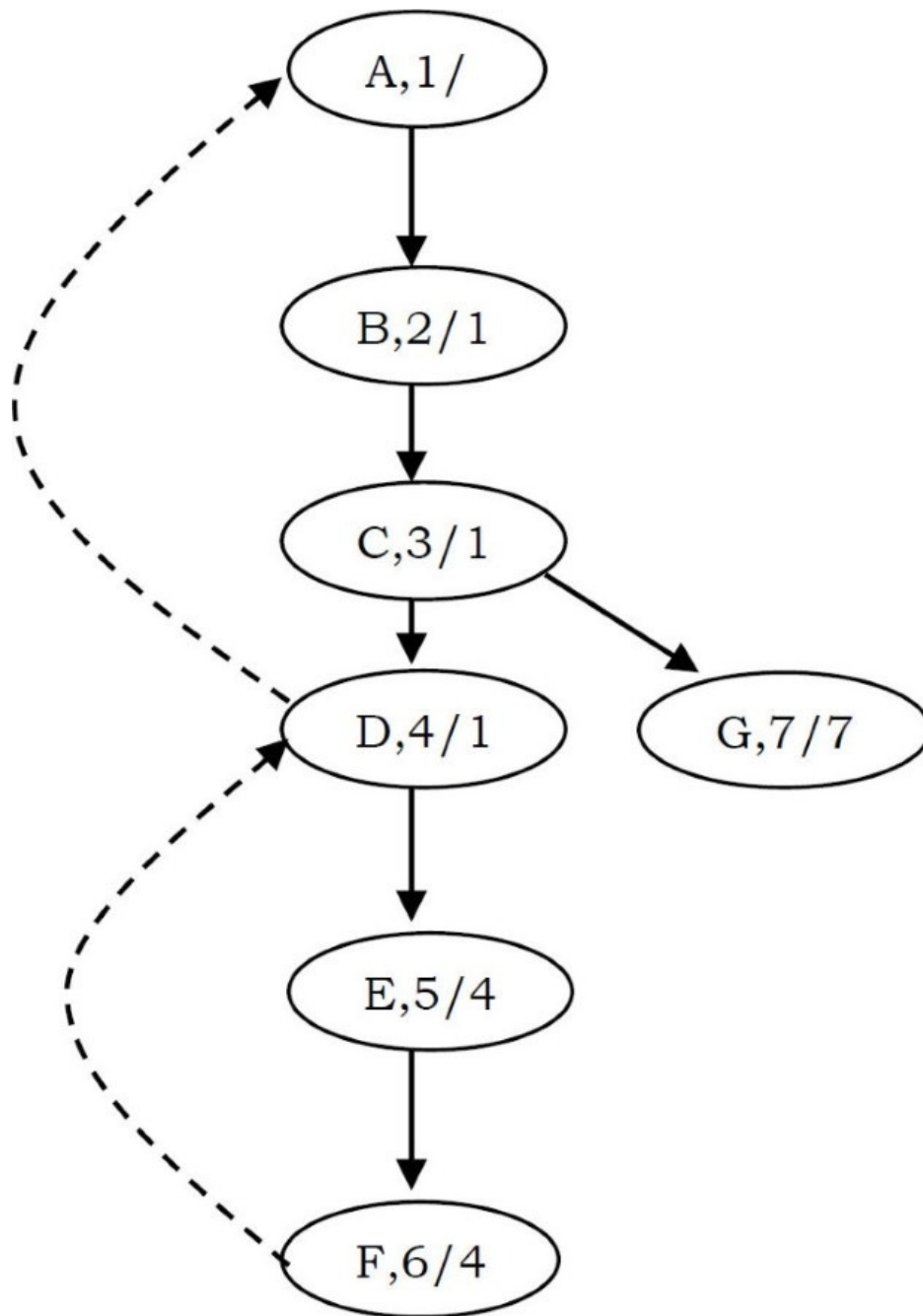


Note: A connected, undirected graph is called *bi – connected* if the graph is still connected after removing any vertex.

DFS provides a linear-time algorithm ($O(n)$) to find all cut vertices in a connected graph. Starting at any vertex, call a *DFS* and number the nodes as they are visited. For each vertex v , we call this *DFS* number $dfsnum(v)$. The tree generated with *DFS* traversal is called *DFS spanning tree*. Then, for every vertex v in the *DFS* spanning tree, we compute the lowest-numbered vertex, which we call $low(v)$, that is reachable from v by taking zero or more tree edges and then possibly one back edge (in that order).

Based on the above discussion, we need the following information for this algorithm: the $dfsnum$ of each vertex in the *DFS* tree (once it gets visited), and for each vertex v , the lowest depth of neighbors of all descendants of v in the *DFS* tree, called the *low*.

The $dfsnum$ can be computed during *DFS*. The *low* of v can be computed after visiting all descendants of v (i.e., just before v gets popped off the *DFS* stack) as the minimum of the $dfsnum$ of all neighbors of v (other than the parent of v in the *DFS* tree) and the *low* of all children of v in the *DFS* tree.



The root vertex is a cut vertex if and only if it has at least two children. A non-root vertex u is a cut vertex if and only if there is a son v of u such that $low(v) \geq dfsnum(u)$. This property can be tested once the *DFS* is returned from every child of u (that means, just before u gets popped off the *DFS* stack), and if true, u separates the graph into different bi-connected components. This can be represented by computing one bi-connected component out of every such v (a component which contains v will contain the sub-tree of v , plus u), and then erasing the sub-tree of v from the tree.

For the given graph, the *DFS* tree with *dfsnum/low* can be given as shown in the figure below. The implementation for the above discussion is:

```

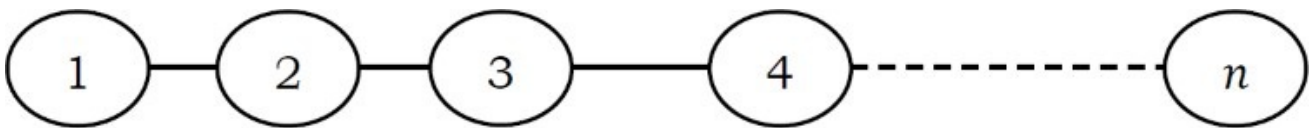
int adjMatrix [256] [256] ;
int dfsnum [256], num = 0, low [256];
void CutVertices( int u ) {
    low[u] = dfsnum[u] = num++;

    for (int v = 0 ; v < 256; ++v ) {
        if(adjMatrix[u][v] && dfsnum[v] == -1) {
            CutVertices( v ) ;
            if(low[v] > dfsnum[u])
                printf("Cut Vetex:%d",u);
            low[u] = min ( low[u] , low[v] ) ;
        }
        else // (u,v) is a back edge
            low[u] = min(low[u] , dfsnum[v]) ;
    }
}

```

Problem-15 Let G be a connected graph of order n . What is the maximum number of cut-vertices that G can contain?

Solution: $n - 2$. As an example, consider the following graph. In the graph below, except for the vertices 1 and n , all the remaining vertices are cut vertices. This is because removing 1 and n vertices does not split the graph into two. This is a case where we can get the maximum number of cut vertices.

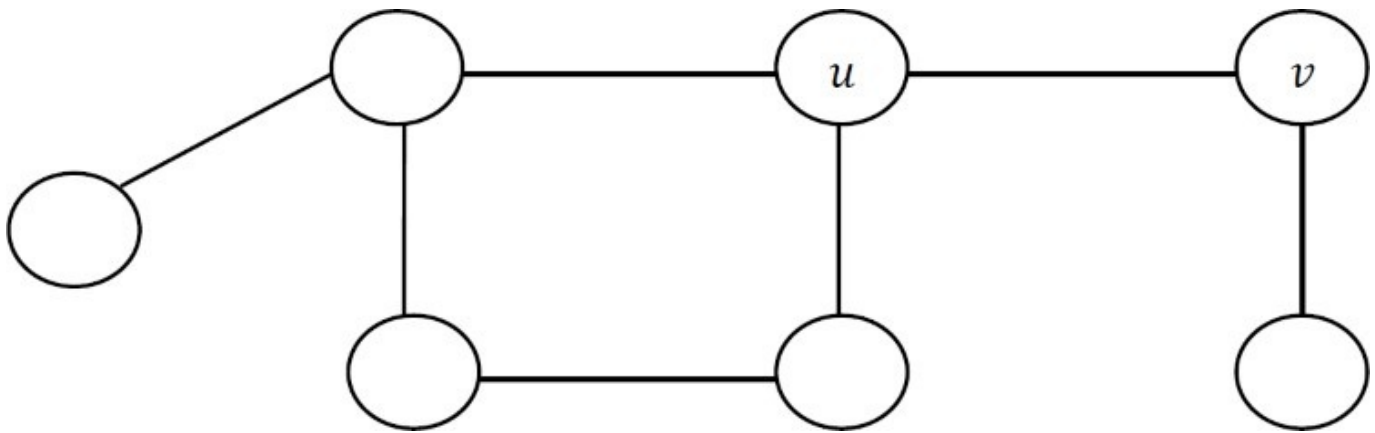


Problem-16 **DFS Application:** *Cut Bridges or Cut Edges*

Solution:

Definition: Let G be a connected graph. An edge uv in G is called a *bridge* of G if $G - uv$ is disconnected.

As an example, consider the following graph.



In the above graph, if we remove the edge uv then the graph splits into two components. For this graph, uv is a bridge. The discussion we had for cut vertices holds good for bridges also. The only change is, instead of printing the vertex, we give the edge. The main observation is that an edge (u, v) cannot be a bridge if it is part of a cycle. If (u, v) is not part of a cycle, then it is a bridge.

We can detect cycles in *DFS* by the presence of back edges, (u, v) is a bridge if and only if none of v or v 's children has a back edge to u or any of u 's ancestors. To detect whether any of v 's children has a back edge to u 's parent, we can use a similar idea as above to see what is the smallest *dfsnum* reachable from the subtree rooted at v .

```
int dfsnum[256], num = 0, low[256];
void Bridges( struct Graph *G, int u ) {
    low[u] = dfsnum[u] = num++;

    for (int v = 0 ; G->V; ++v) {
        if(G->adjMatrix[u][v] && dfsnum[v] == -1) {
            cutVertices( v );

            if(low[v] > dfsnum[u])
                print (u,v) as a bridge

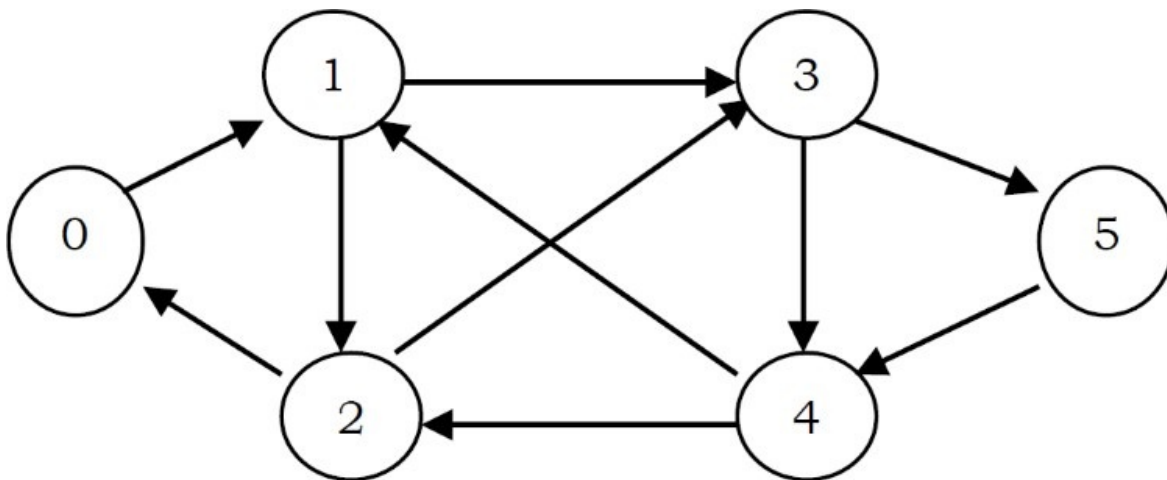
            low[u] = min ( low[u] , low[v] ) ;
        }
        else // (u,v) is a back edge
            low[u] = min(low[u] , dfsnum[v]) ;
    }
}
```

Solution: Before discussing this problem let us see the terminology:

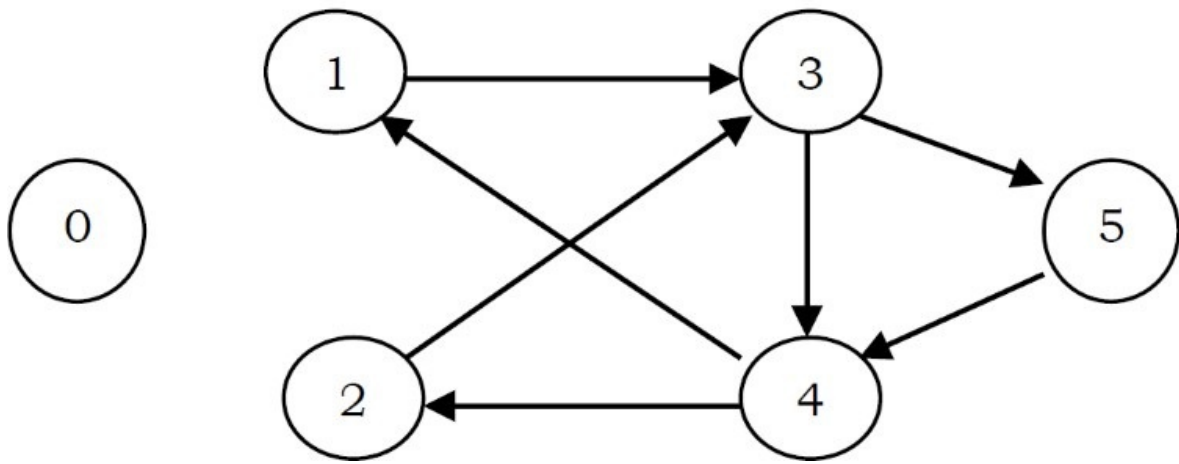
- *Eulerian tour*- a path that contains all edges without repetition.
- *Eulerian circuit* – a path that contains all edges without repetition and starts and ends in the same vertex.
- *Eulerian graph* – a graph that contains an Eulerian circuit.
- *Even vertex*: a vertex that has an even number of incident edges.
- *Odd vertex*: a vertex that has an odd number of incident edges.

Euler circuit: For a given graph we have to reconstruct the circuits using a pen, drawing each line exactly once. We should not lift the pen from the paper while drawing. That means, we must find a path in the graph that visits every edge exactly once and this problem is called an *Euler path* (also called *Euler tour*) or *Euler circuit problem*. This puzzle has a simple solution based on DFS.

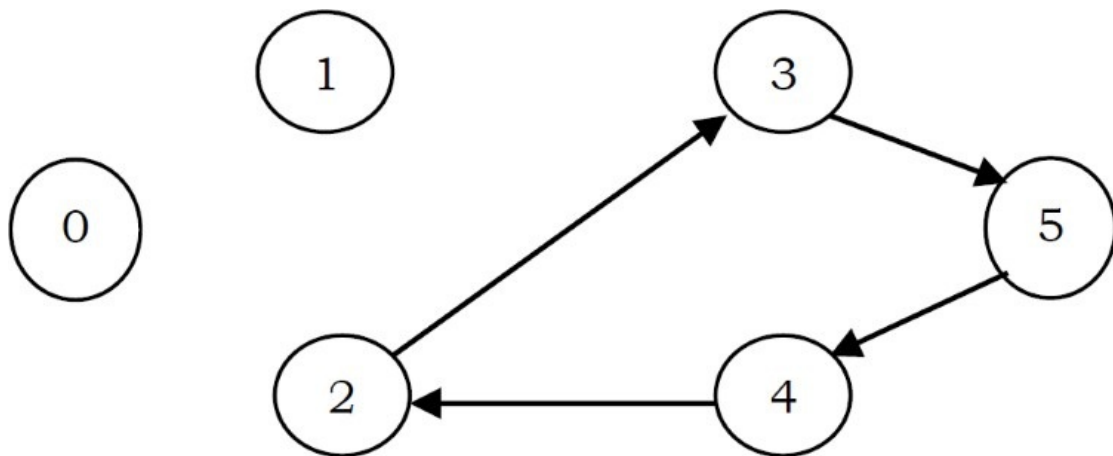
An *Euler circuit* exists if and only if the graph is connected and the number of neighbors of each vertex is even. Start with any node, select any untraversed outgoing edge, and follow it. Repeat until there are no more remaining unselected outgoing edges. For example, consider the following graph: A legal Euler Circuit of this graph is 0 1 3 4 1 2 3 5 4 2 0.



If we start at vertex 0, we can select the edge to vertex 1, then select the edge to vertex 2, then select the edge to vertex 0. There are now no remaining unchosen edges from vertex 0:



We now have a circuit 0,1,2,0 that does not traverse every edge. So, we pick some other vertex that is on that circuit, say vertex 1. We then do another depth first search of the remaining edges. Say we choose the edge to node 3, then 4, then 1. Again we are stuck. There are no more unchosen edges from node 1. We now splice this path 1,3,4,1 into the old path 0,1,2,0 to get: 0,1,3,4,1,2,0. The unchosen edges now look like this:



We can pick yet another vertex to start another DFS. If we pick vertex 2, and splice the path 2,3,5,4,2, then we get the final circuit 0,1,3,4,1,2,3,5,4,2,0.

A similar problem is to find a simple cycle in an undirected graph that visits every vertex. This is known as the *Hamiltonian cycle problem*. Although it seems almost identical to the *Euler* circuit problem, no efficient algorithm for it is known.

Notes:

- A connected undirected graph is *Eulerian* if and only if every graph vertex has an even degree, or exactly two vertices with an odd degree.
- A directed graph is *Eulerian* if it is strongly connected and every vertex has an equal *in* and *out* degree.

Application: A postman has to visit a set of streets in order to deliver mails and packages. He needs to find a path that starts and ends at the post-office, and that passes through each street

(edge) exactly once. This way the postman will deliver mails and packages to all the necessary streets, and at the same time will spend minimum time/effort on the road.

Problem-18 DFS Application: Finding Strongly Connected Components.

Solution: This is another application of DFS. In a directed graph, two vertices u and v are strongly connected if and only if there exists a path from u to v and there exists a path from v to u . The strong connectedness is an equivalence relation.

- A vertex is strongly connected with itself
- If a vertex u is strongly connected to a vertex v , then v is strongly connected to u
- If a vertex u is strongly connected to a vertex v , and v is strongly connected to a vertex x , then u is strongly connected to x

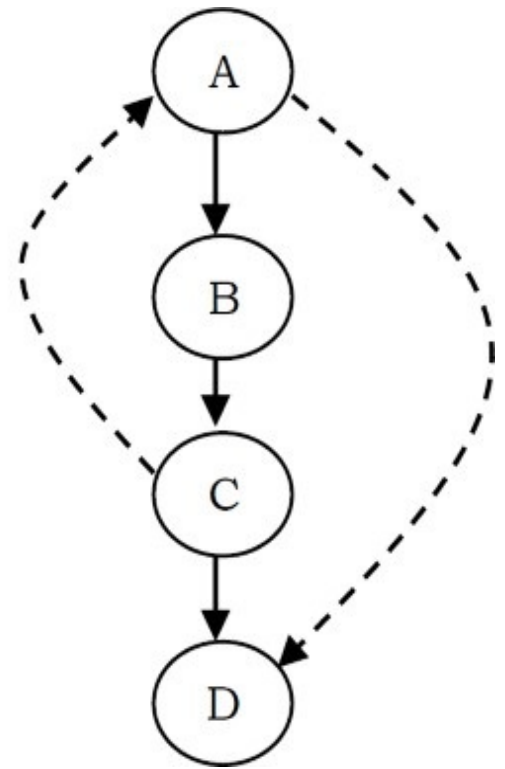
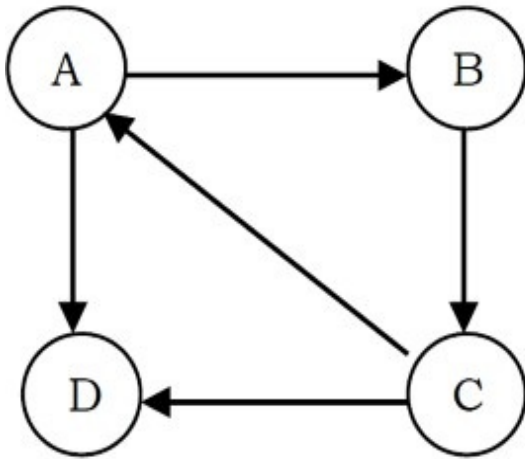
What this says is, for a given directed graph we can divide it into strongly connected components. This problem can be solved by performing two depth-first searches. With two DFS searches we can test whether a given directed graph is strongly connected or not. We can also produce the subsets of vertices that are strongly connected.

Algorithm

- Perform DFS on given graph G .
- Number vertices of given graph G according to a post-order traversal of depth-first spanning forest.
- Construct graph G_r by reversing all edges in G .
- Perform DFS on G_r : Always start a new DFS (initial call to Visit) at the highest-numbered vertex.
- Each tree in the resulting depth-first spanning forest corresponds to a strongly-connected component.

Why this algorithm works?

Let us consider two vertices, v and w . If they are in the same strongly connected component, then there are paths from v to w and from w to v in the original graph G , and hence also in G_r . If two vertices v and w are not in the same depth-first spanning tree of G_r , clearly they cannot be in the same strongly connected component. As an example, consider the graph shown below on the left. Let us assume this graph is G .

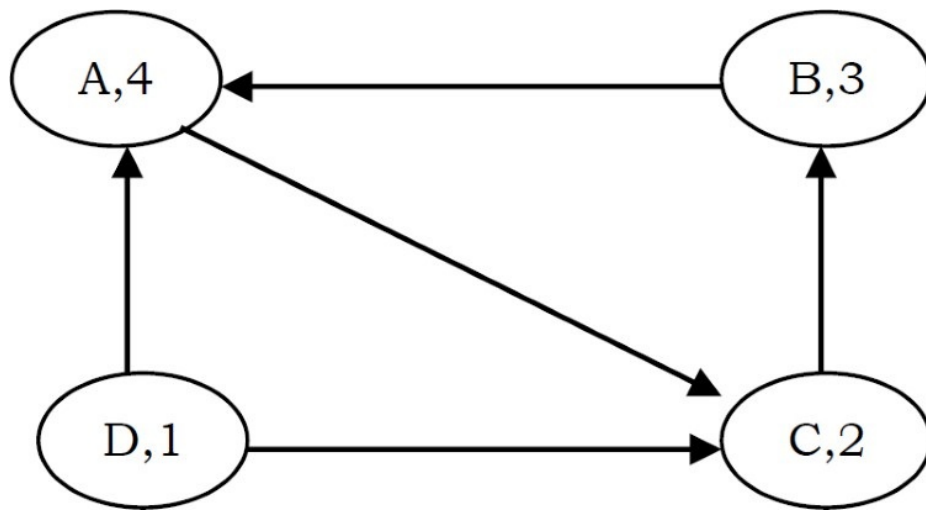


Now, as per the algorithm, performing *DFS* on this G graph gives the following diagram. The dotted line from C to A indicates a back edge.

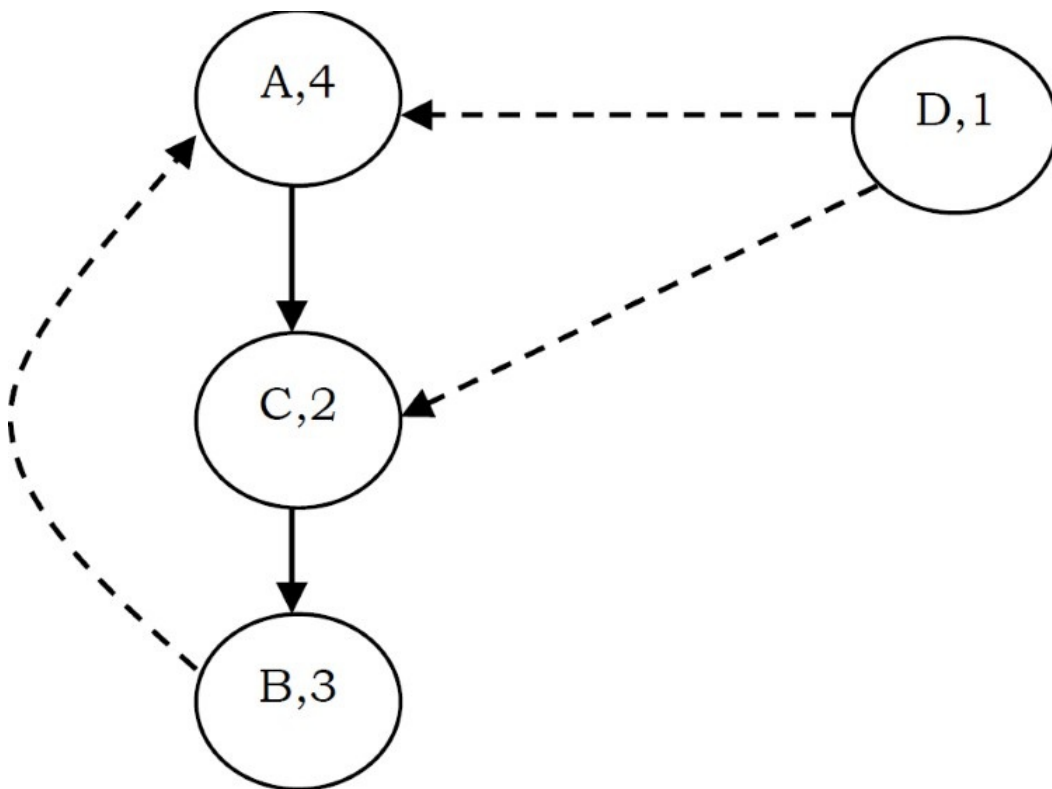
Now, performing post order traversal on this tree gives: D,C,B and A .

Vertex	Post Order Number
A	4
B	3
C	2
D	1

Now reverse the given graph G and call it G_r and at the same time assign postorder numbers to the vertices. The reversed graph G_r will look like:



The last step is performing DFS on this reversed graph G_r . While doing *DFS*, we need to consider the vertex which has the largest DFS number. So, first we start at *A* and with *DFS* we go to *C* and then *B*. At *B*, we cannot move further. This says that $\{A, B, C\}$ is a strongly connected component. Now the only remaining element is *D* and we end our second *DFS* at *D*. So the connected components are: $\{A, B, C\}$ and $\{D\}$.



The implementation based on this discussion can be shown as:

```

//Graph represented in adj matrix.
int adjMatrix [256][256], table[256];
vector <int> st ;
int counter = 0 ;
//This table contains the DFS Search number
int dfsnum [256], num = 0, low[256] ;
void StronglyConnectedComponents( int u ) {
    low[u] = dfsnum[ u ] = num++;
    Push(st, u) ;
    for( int v = 0 ; v < 256; ++v ) {
        if(graph[u][v] && table[v] == -1) {
            if( dfsnum[v] == -1)
                StronglyConnectedComponents(v) ;
            low[u] = min(low[u] , low[v]) ;
        }
    }
    if(low[u] == dfsnum[u]) {
        while( table[u] != counter) {
            table[st.back()] = counter;
            Push(st) ;
        }
        ++ counter;
    }
}
}

```

Problem-19 Count the number of connected components of Graph G which is represented in the adjacent matrix.

Solution: This problem can be solved with one extra counter in *DFS*.

```

//Visited[] is a global array.
int Visited[G→V];
void DFS(struct Graph *G, int u) {
    Visited[u] = 1;
    for( int v = 0; v < G→V; v++ ) {
        /* For example, if the adjacency matrix is used for representing the
           graph, then the condition to be used for finding unvisited adjacent
           vertex of u is: if( !Visited[v] && G→Adj[u][v] ) */
        for each unvisited adjacent node v of u {
            DFS(G, v);
        }
    }
}

void DFSTraversal(struct Graph *G) {
    int count = 0;
    for (int i = 0; i < G→V; i++)
        Visited[i]=0;
    //This loop is required if the graph has more than one component
    for (int i = 0; i < G→V; i++)
        if(!Visited[i]) {
            DFS(G, i);
            count++;
        }

    return count;
}

```

Time Complexity: Same as that of DFS and it depends on implementation. With adjacency matrix the complexity is $O(|E| + |V|)$ and with adjacency matrix the complexity is $O(|V|^2)$.

Problem-20 Can we solve the [Problem-19](#), using BFS?

Solution: Yes. This problem can be solved with one extra counter in BFS.

```

void BFS(struct Graph *G, int u) {
    int v,
    Queue Q = CreateQueue();
    EnQueue(Q, u);
    while(!IsEmptyQueue(Q)) {
        u = DeQueue(Q);
        Process u; //For example, print
        Visited[u]=1;
        /* For example, if the adjacency matrix is used for representing the
           graph, then the condition be used for finding unvisited adjacent
           vertex of u is: if( !Visited[v] && G→Adj[u][v] ) */
        for each unvisited adjacent node v of u {
            EnQueue(Q, v);
        }
    }
}

void BFSTraversal(struct Graph *G) {
    for (int i = 0; i < G→V; i++)
        Visited[i]=0;
    //This loop is required if the graph has more than one component
    for (int i = 0; i < G→V; i++)
        if(!Visited[i])
            BFS(G, i);
}

```

Time Complexity: Same as that of *BFS* and it depends on implementation. With adjacency matrix the complexity is $O(|E| + |V|)$ and with adjacency matrix the complexity is $O(|V|^2)$.

Problem-21 Let us assume that $G(V,E)$ is an undirected graph. Give an algorithm for finding a spanning tree which takes $O(|E|)$ time complexity (not necessarily a minimum spanning tree).

Solution: The test for a cycle can be done in constant time, by marking vertices that have been added to the set S . An edge will introduce a cycle, if both its vertices have already been marked.

Algorithm:

```

S = {}; //Assume S is a set
for each edge e ∈ E {
    if(adding e to S doesn't form a cycle) {
        add e to S;
        mark e;
    }
}

```

Problem-22 Is there any other way of solving 0?

Solution: Yes. We can run *BFS* and find the *BFS* tree for the graph (level order tree of the graph). Then start at the root element and keep moving to the next levels and at the same time we have to consider the nodes in the next level only once. That means, if we have a node with multiple input edges then we should consider only one of them; otherwise they will form a cycle.

Problem-23 Detecting a cycle in an undirected graph

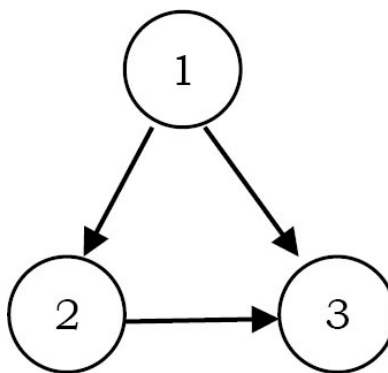
Solution: An undirected graph is acyclic if and only if a *DFS* yields no back edges, edges (u, v) where v has already been discovered and is an ancestor of u .

- Execute *DFS* on the graph.
- If there is a back edge – the graph has a cycle.

If the graph does not contain a cycle, then $|E| < |V|$ and *DFS* cost $O(|V|)$. If the graph contains a cycle, then a back edge is discovered after $2|V|$ steps at most.

Problem-24 Detecting a cycle in DAG

Solution:



Cycle detection on a graph is different than on a tree. This is because in a graph, a node can have multiple parents. In a tree, the algorithm for detecting a cycle is to do a depth first search, marking nodes as they are encountered. If a previously marked node is seen again, then a cycle exists. This won't work on a graph. Let us consider the graph shown in the figure below. If we use a tree cycle detection algorithm, then it will report the wrong result. That means that this graph has a cycle in it. But the given graph does not have a cycle in it. This is because node 3 will be seen twice in a *DFS* starting at node 1.

The cycle detection algorithm for trees can easily be modified to work for graphs. The key is that in a *DFS* of an acyclic graph, a node whose descendants have all been visited can be seen again without implying a cycle. But, if a node is seen for the second time before all its descendants have been visited, then there must be a cycle. Can you see why this is? Suppose there is a cycle containing node A. This means that A must be reachable from one of its descendants. So when the *DFS* is visiting that descendant, it will see A again, before it has finished visiting all of A's descendants. So there is a cycle. In order to detect cycles, we can modify the depth first search.

```
int DetectCycle(struct Graph *G) {
    for (int i = 0; i < G→V; i++) {
        Visited[s]=0;
        Predecessor[i] = 0;
    }
    for (int i = 0; i < G→V; i++) {
        if(!Visited[i] && HasCycle(G, i))
            return 1;
    }
    return false;;
}

int HasCycle(struct Graph *G, int u) {
    Visited[u]=1;
    for (int i = 0; i < G→V; i++) {
        if(G→Adj[s][i]) {
            if(Predecessor[i] != u && Visited[i])
                return 1;
            else {
                Predecessor[i] = u;
                return HasCycle(G, i);
            }
        }
    }
    return 0;
}
```

Time Complexity: $O(V + E)$.

Problem-25 Given a directed acyclic graph, give an algorithm for finding its depth.

Solution: If it is an undirected graph, we can use the simple unweighted shortest path algorithm (check *Shortest Path Algorithms* section). We just need to return the highest number among all distances. For directed acyclic graph, we can solve by following the similar approach which we used for finding the depth in trees. In trees, we have solved this problem using level order

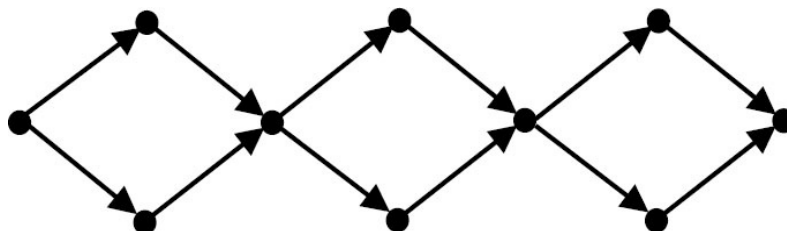
traversal (with one extra special symbol to indicate the end of the level).

```
// Assuming the given graph is a DAG
int DepthInDAG( struct Graph *G ) {
    struct Queue *Q;
    int counter;
    int v, w;
    Q = CreateQueue();

    counter = 0;
    for (v = 0; v < G→V; v++)
        if( indegree[v] == 0 )
            EnQueue( Q , v );
    EnQueue( Q, '$' );
    while( !IsEmptyQueue( Q ) ) {
        v = DeQueue( Q );
        if( v == '$' ) {
            counter++;
            if( !IsEmptyQueue( Q ) )
                EnQueue( Q, '$' );
        }
        for each w adjacent to v
            if( --indegree[w] == 0 )
                EnQueue( Q , w );
    }
    DeleteQueue( Q );
    return counter;
}
```

Total running time is $O(V + E)$.

Problem-26 How many topological sorts of the following dag are there?



Solution: If we observe the above graph there are three stages with 2 vertices. In the early

discussion of this chapter, we saw that topological sort picks the elements with zero indegree at any point of time. At each of the two vertices stages, we can first process either the top vertex or the bottom vertex. As a result, at each of these stages we have two possibilities. So the total number of possibilities is the multiplication of possibilities at each stage and that is, $2 \times 2 \times 2 = 8$.

Problem-27 Unique topological ordering: Design an algorithm to determine whether a directed graph has a unique topological ordering.

Solution: A directed graph has a unique topological ordering if and only if there is a directed edge between each pair of consecutive vertices in the topological order. This can also be defined as: a directed graph has a unique topological ordering if and only if it has a Hamiltonian path. If the digraph has multiple topological orderings, then a second topological order can be obtained by swapping a pair of consecutive vertices.

Problem-28 Let us consider the prerequisites for courses at *IIT Bombay*. Suppose that all prerequisites are mandatory, every course is offered every semester, and there is no limit to the number of courses we can take in one semester. We would like to know the minimum number of semesters required to complete the major. Describe the data structure we would use to represent this problem, and outline a linear time algorithm for solving it.

Solution: Use a directed acyclic graph (DAG). The vertices represent courses and the edges represent the prerequisite relation between courses at *IIT Bombay*. It is a DAG, because the prerequisite relation has no cycles.

The number of semesters required to complete the major is one more than the longest path in the dag. This can be calculated on the DFS tree recursively in linear time. The longest path out of a vertex x is 0 if x has outdegree 0, otherwise it is $1 + \max \{ \text{longest path out of } y \mid (x,y) \text{ is an edge of } G \}$.

Problem-29 At a university let's say *IIT Bombay*), there is a list of courses along with their prerequisites. That means, two lists are given:

A – Courses list

B – Prerequisites: B contains couples (x,y) where $x,y \in A$ indicating that course x can't be taken before course y .

Let us consider a student who wants to take only one course in a semester. Design a schedule for this student.

Example: $A = \{\text{C-Lang, Data Structures, OS, CO, Algorithms, Design Patterns, Programming}\}$. $B = \{ (\text{C-Lang, CO}), (\text{OS, CO}), (\text{Data Structures, Algorithms}), (\text{Design Patterns, Programming}) \}$. One possible schedule could be:

Semester 1:	Data Structures
Semester 2:	Algorithms
Semester 3:	C-Lang

Semester 4:	OS
Semester 5:	CO
Semester 6:	Design Patterns
Semester 7:	Programming

Solution: The solution to this problem is exactly the same as that of topological sort. Assume that the courses names are integers in the range $[1..n]$, n is known (n is not constant). The relations between the courses will be represented by a directed graph $G = (V, E)$, where V are the set of courses and if course i is prerequisite of course j , E will contain the edge (i, j) . Let us assume that the graph will be represented as an Adjacency list.

First, let's observe another algorithm to topologically sort a DAG in $O(|V| + |E|)$.

- Find in-degree of all the vertices - $O(|V| + |E|)$
- Repeat:
 - Find a vertex v with in-degree=0 - $O(|V|)$
 - Output v and remove it from G , along with its edges - $O(|V|)$
 - Reduce the in-degree of each node u such as (v, u) was an edge in G and keep a list of vertices with in-degree=0 – $O(\text{degree}(v))$
 - Repeat the process until all the vertices are removed

The time complexity of this algorithm is also the same as that of the topological sort and it is $O(|V| + |E|)$.

Problem-30 In [Problem-29](#), a student wants to take all the courses in A , in the minimal number of semesters. That means the student is ready to take any number of courses in a semester. Design a schedule for this scenario. *One possible schedule is:*
 Semester 1: C-Lang, OS, Design Patterns
 Semester 2: Data Structures, CO, Programming
 Semester 3: Algorithms

Solution: A variation of the above topological sort algorithm with a slight change: In each semester, instead of taking one subject, take all the subjects with zero indegree. That means, execute the algorithm on all the nodes with degree 0 (instead of dealing with one source in each stage, all the sources will be dealt and printed).

Time Complexity: $O(|V| + |E|)$.

Problem-31 LCA of a DAG: Given a DAG and two vertices v and w , find the *lowest common ancestor* (LCA) of v and w . The LCA of v and w is an ancestor of v and w that has no descendants that are also ancestors of v and w .

Hint: Define the height of a vertex v in a DAG to be the length of the longest path from *root* to v . Among the vertices that are ancestors of both v and w , the one with the greatest height is an LCA

of v and w .

Problem-32 Shortest ancestral path: Given a DAG and two vertices v and w , find the *shortest ancestral path* between v and w . An ancestral path between v and w is a common ancestor x along with a shortest path from v to x and a shortest path from w to x . The shortest ancestral path is the ancestral path whose total length is minimized.

Hint: Run BFS two times. First run from v and second time from w . Find a DAG where the shortest ancestral path goes to a common ancestor x that is not an LCA.

Problem-33 Let us assume that we have two graphs G_1 and G_2 . How do we check whether they are isomorphic or not?

Solution: There are many ways of representing the same graph. As an example, consider the following simple graph. It can be seen that all the representations below have the same number of vertices and the same number of edges.



Definition: Graphs $G_1 = \{V_1, E_1\}$ and $G_2 = \{V_2, E_2\}$ are isomorphic if

- 1) There is a one-to-one correspondence from V_1 to V_2 and
- 2) There is a one-to-one correspondence from E_1 to E_2 that map each edge of G_1 to G_2 .

Now, for the given graphs how do we check whether they are isomorphic or not?

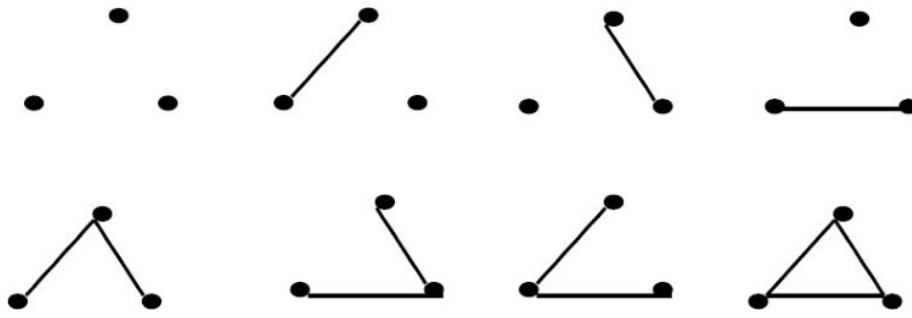
In general, it is not a simple task to prove that two graphs are isomorphic. For that reason we must consider some properties of isomorphic graphs. That means those properties must be satisfied if the graphs are isomorphic. If the given graph does not satisfy these properties then we say they are not isomorphic graphs.

Property: Two graphs are isomorphic if and only if for some ordering of their vertices their adjacency matrices are equal.

Based on the above property we decide whether the given graphs are isomorphic or not. In order to check the property, we need to do some matrix transformation operations.

Problem-34 How many simple undirected non-isomorphic graphs are there with n vertices?

Solution: We will try to answer this question in two steps. First, we count all labeled graphs. Assume all the representations below are labeled with $\{1,2,3\}$ as vertices. The set of all such graphs for $n = 3$ are:



There are only two choices for each edge: it either exists or it does not. Therefore, since the maximum number of edges is $\binom{n}{2}$ (and since the maximum number of edges in an undirected graph with n vertices is $\frac{n(n-1)}{2} = n_{c_2} = \binom{n}{2}$), the total number of undirected labeled graphs is $2^{\binom{n}{2}}$.

Problem-35 Hamiltonian path in DAGs: Given a DAG, design a linear time algorithm to determine whether there is a path that visits each vertex exactly once.

Solution: The *Hamiltonian* path problem is an NP-Complete problem (for more details ref *Complexity Classes* chapter). To solve this problem, we will try to give the approximation algorithm (which solves the problem, but it may not always produce the optimal solution).

Let us consider the topological sort algorithm for solving this problem. Topological sort has an interesting property: that if all pairs of consecutive vertices in the sorted order are connected by edges, then these edges form a directed *Hamiltonian* path in the DAG. If a *Hamiltonian* path exists, the topological sort order is unique. Also, if a topological sort does not form a *Hamiltonian* path, the DAG will have two or more topological orderings.

Approximation Algorithm: Compute a topological sort and check if there is an edge between each consecutive pair of vertices in the topological order.

In an unweighted graph, find a path from s to t that visits each vertex exactly once. The basic solution based on backtracking is, we start at s and try all of its neighbors recursively, making sure we never visit the same vertex twice. The algorithm based on this implementation can be given as:

```

bool seenTable[32];
void HamiltonianPath( struct Graph *G, int u ) {
    if( u == t )
        /* Check that we have seen all vertices. */
    else {
        for( int v = 0; v < n; v++ )
            if( !seenTable[v] && G->Adj[u][v] ) {
                seenTable[v] = true;
                HamiltonianPath( v );
                seenTable[v] = false;
            }
    }
}

```

Note that if we have a partial path from s to u using vertices $s = v_1, v_2, \dots, v_k = u$, then we don't care about the order in which we visited these vertices so as to figure out which vertex to visit next. All that we need to know is the set of vertices we have seen (the `seenTable[]` array) and which vertex we are at right now (u). There are 2^n possible sets of vertices and n choices for u . In other words, there are 2^n possible `seenTable[]` arrays and n different parameters to `Hamiltonian_path()`. What `Hamiltonian_path()` does during any particular recursive call is completely determined by the `seenTable[]` array and the parameter u .

Problem-36 For a given graph G with n vertices how many trees we can construct?

Solution: There is a simple formula for this problem and it is named after Arthur Cayley. For a given graph with n labeled vertices the formula for finding number of trees on is n^{n-2} . Below, the number of trees with different n values is shown.

n value	Formula value: n^{n-2}	Number of Trees
2	1	1 ————— 2
3	3	1 2 3 3 1 2 2 3 1

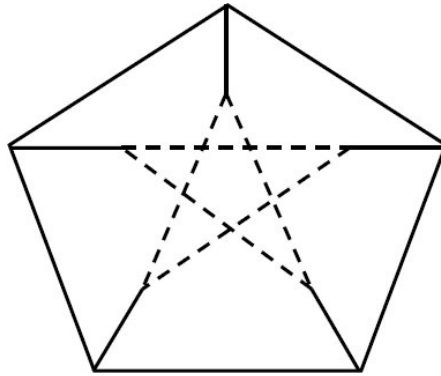
Problem-37 For a given graph G with n vertices how many spanning trees can we construct?

Solution: The solution to this problem is the same as that of [Problem-36](#). It is just another way of asking the same question. Because the number of edges in both regular tree and spanning tree are the same.

Problem-38 The *Hamiltonian cycle* problem: Is it possible to traverse each of the vertices of a graph exactly once, starting and ending at the same vertex?

Solution: Since the *Hamiltonian* path problem is an NP-Complete problem, the *Hamiltonian* cycle problem is an NP-Complete problem. A *Hamiltonian* cycle is a cycle that traverses every vertex of a graph exactly once. There are no known conditions in which are both necessary and sufficient, but there are a few sufficient conditions.

- For a graph to have a *Hamiltonian* cycle the degree of each vertex must be two or more.
- The Petersen graph does not have a *Hamiltonian* cycle and the graph is given below.



- In general, the more edges a graph has, the more likely it is to have a *Hamiltonian* cycle.
- Let G be a simple graph with $n \geq 3$ vertices. If every vertex has a degree of at least $\frac{n}{2}$, then G has a *Hamiltonian* cycle.
- The best known algorithm for finding a *Hamiltonian* cycle has an exponential worst-case complexity.

Note: For the approximation algorithm of *Hamiltonian* path, refer to the [Dynamic Programming](#) chapter.

Problem-39 What is the difference between *Dijkstra's* and *Prim's* algorithm?

Solution: *Dijkstra's* algorithm is almost identical to that of *Prim's*. The algorithm begins at a specific vertex and extends outward within the graph until all vertices have been reached. The only distinction is that *Prim's* algorithm stores a minimum cost edge whereas *Dijkstra's* algorithm stores the total cost from a source vertex to the current vertex. More simply, *Dijkstra's* algorithm stores a summation of minimum cost edges whereas *Prim's* algorithm stores at most one minimum cost edge.

Problem-40 **Reversing Graph:** : Give an algorithm that returns the reverse of the directed graph (each edge from v to w is replaced by an edge from w to v).

Solution: In graph theory, the reverse (also called *transpose*) of a directed graph G is another directed graph on the same set of vertices with all the edges reversed. That means, if G contains an edge (u, v) then the reverse of G contains an edge (v, u) and vice versa.

Algorithm:

```
Graph ReverseTheDirectedGraph(struct Graph *G) {  
    Create new graph with name ReversedGraph and  
        let us assume that this will contain the reversed graph.  
    //The reversed graph also will contain same number of vertices and edges.  
    for each vertex of given graph G {  
        for each vertex w adjacent to v {  
            Add the w to v edge in ReversedGraph;  
            //That means we just need to reverse the bits in adjacency matrix.  
        }  
    }  
    return ReversedGraph;  
}
```

Problem-41 Travelling Sales Person Problem: Find the shortest path in a graph that visits each vertex at least once, starting and ending at the same vertex?

Solution: The Traveling Salesman Problem (*TSP*) is related to finding a Hamiltonian cycle. Given a weighted graph G , we want to find the shortest cycle (may be non-simple) that visits all the vertices.

Approximation algorithm: This algorithm does not solve the problem but gives a solution which is within a factor of 2 of optimal (in the worst-case).

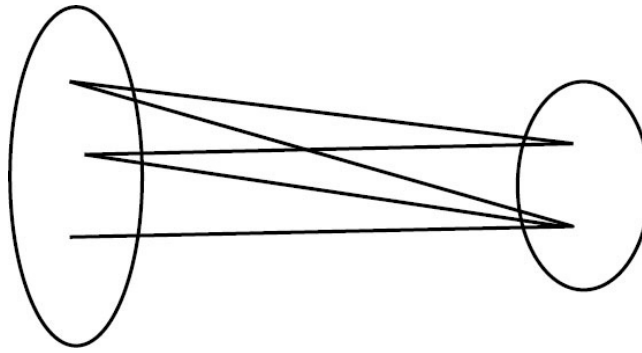
- 1) Find a Minimal Spanning Tree (MST).
- 2) Do a DFS of the MST.

For details, refer to the chapter on [Complexity Classes](#).

Problem-42 Discuss Bipartite matchings?

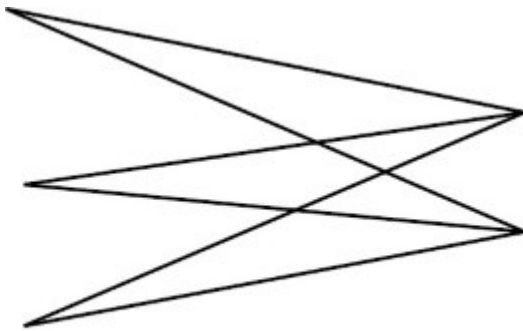
Solution: In Bipartite graphs, we divide the graphs into two disjoint sets, and each edge connects a vertex from one set to a vertex in another subset (as shown in figure).

Definition: A simple graph $G = (V, E)$ is called a *bipartite graph* if its vertices can be divided into two disjoint sets $V = V_1 \cup V_2$, such that every edge has the form $e = (a, b)$ where $a \in V_1$ and $b \in V_2$. One important condition is that no vertices both in V_1 or both in V_2 are connected.

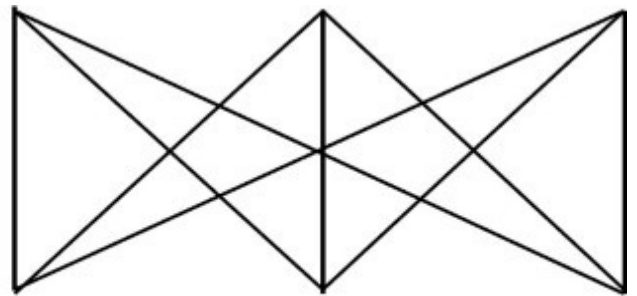


Properties of Bipartite Graphs

- A graph is called bipartite if and only if the given graph does not have an odd length cycle.
- A *complete bipartite graph* $K_{m,n}$ is a bipartite graph that has each vertex from one set adjacent to each vertex from another set.

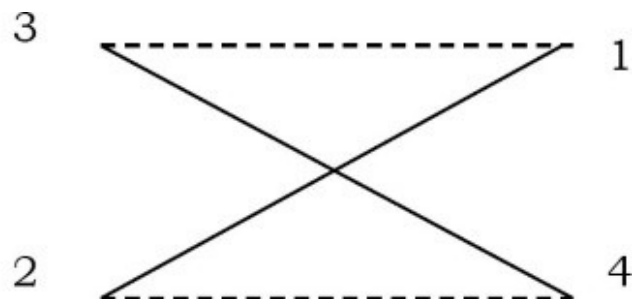


$K_{2,3}$



$K_{3,3}$

- A subset of edges $M \subset E$ is a *matching* if no two edges have a common vertex. As an example, matching sets of edges are represented with dotted lines. A matching M is called *maximum* if it has the largest number of possible edges. In the graphs, the dotted edges represent the alternative matching for the given graph.



- A matching M is *perfect* if it matches all vertices. We must have $V_1 = V_2$ in order to have perfect matching.
- An *alternating path* is a path whose edges alternate between matched and unmatched edges. If we find an alternating path, then we can improve the matching. This is because an alternating path consists of matched and unmatched edges. The number of unmatched edges exceeds the number of matched edges by one.

Therefore, an alternating path always increases the matching by one.

The next question is, how do we find a perfect matching? Based on the above theory and definition, we can find the perfect matching with the following approximation algorithm.

Matching Algorithm (Hungarian algorithm)

- 1) Start at unmatched vertex.
- 2) Find an alternating path.
- 3) If it exists, change matching edges to no matching edges and conversely. If it does not exist, choose another unmatched vertex.
- 4) If the number of edges equals $V/2$, stop. Otherwise proceed to step 1 and repeat, as long as all vertices have been examined without finding any alternating paths.

Time Complexity of the Matching Algorithm: The number of iterations is in $O(V)$. The complexity of finding an alternating path using BFS is $O(E)$. Therefore, the total time complexity is $O(V \times E)$.

Problem-43 Marriage and Personnel Problem?

Marriage Problem: There are X men and Y women who desire to get married. Participants indicate who among the opposite sex could be a potential spouse for them. Every woman can be married to at most one man, and every man to at most one woman. How can we marry everybody to someone they like?

Personnel Problem: You are the boss of a company. The company has M workers and N jobs. Each worker is qualified to do some jobs, but not others. How will you assign jobs to each worker?

Solution: These two cases are just another way of asking about bipartite graphs, and the solution is the same as that of [Problem-42](#).

Problem-44 How many edges will be there in complete bipartite graph $K_{m,n}$?

Solution: $m \times n$. This is because each vertex in the first set can connect all vertices in the second set.

Problem-45 A graph is called a regular graph if it has no loops and multiple edges where each vertex has the same number of neighbors; i.e., every vertex has the same degree. Now, if $K_{m,n}$ is a regular graph, what is the relation between m and n ?

Solution: Since each vertex should have the same degree, the relation should be $m = n$.

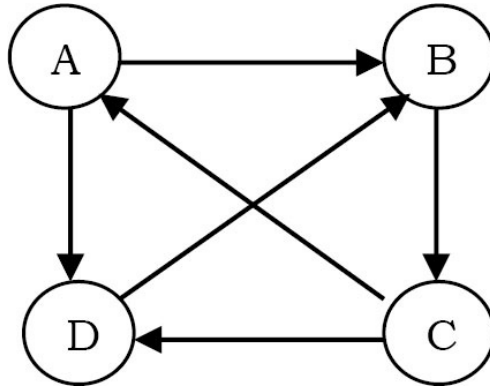
Problem-46 What is the maximum number of edges in the maximum matching of a bipartite graph with n vertices?

Solution: From the definition of *matching*, we should not have edges with common vertices. So

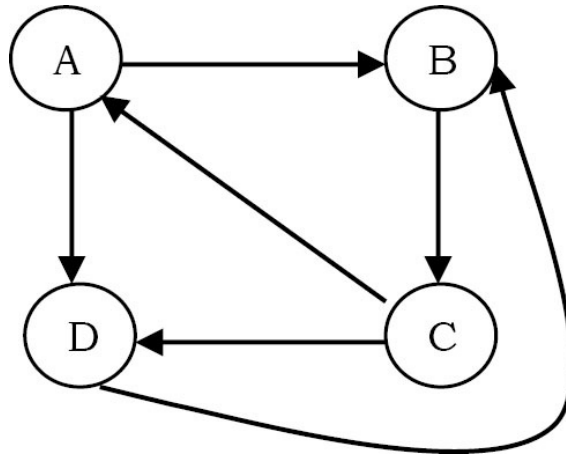
in a bipartite graph, each vertex can connect to only one vertex. Since we divide the total vertices into two sets, we can get the maximum number of edges if we divide them in half. Finally the answer is $\frac{n}{2}$.

Problem-47 Discuss Planar Graphs. *Planar graph*: Is it possible to draw the edges of a graph in such a way that the edges do not cross?

Solution: A graph G is said to be planar if it can be drawn in the plane in such a way that no two edges meet each other except at a vertex to which they are incident. Any such drawing is called a plane drawing of G . As an example consider the below graph:



This graph we can easily convert to a planar graph as below (without any crossed edges).



How do we decide whether a given graph is planar or not?

The solution to this problem is not simple, but researchers have found some interesting properties that we can use to decide whether the given graph is a planar graph or not.

Properties of Planar Graphs

- If a graph G is a connected planar simple graph with V vertices, where $V = 3$ and E edges, then $E = 3V - 6$.
- K_5 is non-planar. [K_5 stands for complete graph with 5 vertices].
- If a graph G is a connected planar simple graph with V vertices and E edges, and no